

EFREI PARIS - M1 DATA ENGINEERING & IA

Urban Data Explorer

150 Questions de Soutenance avec Reponses Detaillees

Bloc 1 - Competences C1.1 a C2.4

Adam Beloucif & Emilien Morice

Soutenance - Juin 2026

150 Questions - Reponses Detaillees - Preparation Jury

Section 1 - Architecture generale et choix techniques

C1.1 Architecture

1 Decrivez l'architecture globale d'Urban Data Explorer.

UDE repose sur trois couches complementaires : (1) PostgreSQL en schema etoile pour les requetes analytiques OLAP, (2) Cassandra en modele query-first pour les donnees d'evenements haute frequence avec partitionnement par event_type et TTL de 7 jours, (3) Kafka pour le streaming micro-batch toutes les 10 secondes. Le frontend MapLibre consomme une API FastAPI securisee par JWT. Le pipeline de collecte ingere 24 sources issues de 8 familles de donnees urbaines.

C1.1 Choix BD

2 Pourquoi avoir choisi trois bases de donnees differentes ?

Chaque base repond a un cas d'usage distinct. PostgreSQL est optimise pour les jointures complexes et les agregations sur des tables de faits/dimensions - ideal pour les indicateurs composites. Cassandra offre une ecriture lineairement scalable et une lecture ultra-rapide par partition, parfaite pour les flux d'evenements urbains (incidents, trafic temps reel). Kafka sert de bus d'evenements qui decouple les producteurs de donnees des consommateurs, garantissant la durabilite et le rejeu.

C1.1 Schema

3 Quel est le schema en etoile utilise dans PostgreSQL ?

Le schema comporte une table de faits fact_events (event_id, event_type, timestamp, localisation_id, value, source_id) et plusieurs tables de dimensions : dim_localisation (IRIS, commune, departement, coordonnees), dim_time (annee, mois, jour, heure, jour_semaine), dim_source (nom, famille, url, frequence), dim_event_type (categorie, unite, agregation_default). Ce modele permet des requetes GROUP BY multidimensionnelles efficaces via index partiels sur les colonnes les plus filtrees.

C1.2 Kafka

4 Comment fonctionne le micro-batch Kafka toutes les 10 secondes ?

Un consumer Kafka Python lit les messages du topic urban_events par lot de 10 secondes (poll(timeout_ms=10000, max_records=5000)). Chaque batch est transforme via Polars (validation schema, deduplication par event_id), puis ecrit en bulk dans Cassandra via batch statements UNLOGGED (limite 100 inserts/batch pour eviter les coordinations distribuees) et en PostgreSQL via COPY FROM pour les evenements qualifies. Le lag consumer est monitore via JMX.

C1.2 Polars

5 Pourquoi Polars plutot que Pandas dans ce projet ?

Polars offre une execution multi-threaded native sur CPU via Apache Arrow, sans GIL Python. Pour un pipeline traitant des fichiers CSV de plusieurs centaines de MB (24 sources), la difference de vitesse est 5-10x. La syntaxe lazy (scan_csv, filter, groupby, collect) permet de construire un plan d'execution optimise avant toute lecture disque. De plus, Polars integre nativement la jointure spatiale avec Shapely pour l'appariement IRIS - essentiel pour aggregator les evenements par zone geographique.

C1.2 IRIS

6 Expliquez la jointure spatiale Polars + Shapely pour l'IRIS.

Chaque evenement arrive avec des coordonnees GPS (lat, lon). On charge le fond de carte IRIS (GeoJSON, 36 000+ geometries) en GeoDataFrame Shapely. La jointure consiste a appliquer un STRtree (Sorted-Tile R-tree) pour chercher le polygone IRIS contenant chaque point en $O(\log n)$. Cet enrichissement transforme lat/lon bruts en code IRIS INSEE (9 chiffres), permettant l'agregation par zone infra-communale normalisee. Le STRtree est construit une fois en memoire au demarrage du pipeline.

C1.3 API

7 Comment avez-vous structure l'API FastAPI ?

L'API est organisee en routers par domaine : /events (CRUD), /indicators (calculs agregats), /map (GeoJSON pour MapLibre), /auth (JWT). Chaque endpoint utilise l'injection de dependances FastAPI pour la connexion BD (asynpg pour PostgreSQL, cassandra-driver async pour Cassandra). Le middleware JWT valide le Bearer token sur chaque requete protegee. Les schemas Pydantic v2 garantissent la validation entree/sortie. La documentation OpenAPI est auto-generee via /docs.

C1.3 JWT

8 Quels sont les quotas JWT et comment sont-ils implements ?

Deux niveaux de quotas : 120 req/min pour les utilisateurs standard, 600 req/min pour les utilisateurs premium. Ils sont implements via un middleware Redis avec la cle `rate:{user_id}:{minute_epoch}`. A chaque requete, on INCR la cle et on EXPIRE a 60s. Si le compteur depasse le seuil, on retourne HTTP 429 avec le header `Retry-After`. Le choix Redis (in-memory, $O(1)$) permet de gerer des milliers de requetes par seconde sans latence notable sur l'API.

C1.1 Sources

9 Quelles sont les 4 familles principales de sources de donnees ?

Parmi les 8 familles : (1) Mobilite - comptages trafic DIRIF, velib, RATP GTFS, (2) Environnement - stations Airparif NO2/PM2.5, meteo Meteoconcept, (3) Securite - incidents BSPP (Brigade de Sapeurs-Pompiers de Paris), main courante police, (4) Social-economique - DVF transactions immobilieres, fichier Sirene entreprises, revenus FiLoSoFi INSEE. Chaque famille a un adaptateur Python dedie qui normalise vers le schema interne avant ingestion Kafka.

C1.1 Fraicheur

10 Comment est geree la fraicheur des 24 sources de donnees ?

Chaque source a une frequence declaree dans `dim_source` : temps-reel (Kafka direct), horaire (cron + API REST), quotidien (cron 02:00, batch FTP/CSV), hebdomadaire/mensuel (OpenData static). Le pipeline verifie via une table `metadata_freshness` (`source_id`, `last_fetched`, `expected_interval`) et leve une alerte si `last_fetched > expected_interval * 1.5`. Les sources critiques (trafic, qualite air) ont une alerte a 10 minutes de retard.

C2.1 Indicateurs

11 Quels sont les 4 indicateurs composites calcules ?

1) Indice Vitalite Urbaine (IVU) : combine trafic, commerce actif Sirene, evenements culturels, mobilite douce, notes 0-100. 2) Score Environnemental : NO2, PM2.5, bruit (dB), espaces verts m2/habitant, pondere par seuils OMS. 3) Indice Securite Relative : incidents/1000 habitants, normalise par densite, lisse sur 30 jours. 4) Accessibilite Multimodale : stations TC dans rayon 500m, score 0-10 par zone IRIS. Ces indices sont pre-calcules toutes les heures et stockes en table agregee pour eviter les requetes lourdes.

C2.2 Carto

12 Comment est visualisee la carte dans MapLibre ?

MapLibre GL JS charge les tuiles vectorielles depuis un serveur pmtiles (protocole compact, single-file). Les zones IRIS sont colorees par choroplethie selon l'indicateur selectionne (echelle de couleur divergente ou sequentielle selon la semantique). Les evenements ponctuels (incidents, capteurs) sont representes en clusters automatiques. Un panneau lateral affiche le detail IRIS au clic : sparklines des 30 derniers jours par indicateur, histogramme des sources, lien vers les donnees brutes. Le rendu WebGL permet l'affichage fluide de 36 000 polygones.

C2.3 Tests

13 Pourquoi 102 tests pytest et que couvrent-ils ?

102 tests repartis : 34 tests unitaires (validateurs Pydantic, calcul indicateurs, clean IRIS), 28 tests integration BD (fixtures PostgreSQL in-memory via testcontainers, Cassandra local), 22 tests API (FastAPI TestClient, tous les endpoints CRUD + erreurs 401/422/429), 18 tests pipeline Kafka (mock consumer, batch Polars), comportement aux limites (null, coordonnees hors France, sources indisponibles). Couverture mesuree : 87% lignes. CI GitHub Actions lance pytest + coverage a chaque push.

C2.4 Limites

14 Quelles sont les limites actuelles du projet ?

Quatre limites principales : (1) Pas de haute disponibilite - Cassandra en single-node, pas de replication factor > 1 en dev. (2) La jointure IRIS relance le STRtree entier si la memoire swap - non resolu en production. (3) Certaines sources OpenData (DVF, Sirene) ont des retards de mise a jour de 3-6 mois cote source officielle, rendant les indicateurs socio-eco partiellement obsoletes. (4) L'API ne supporte pas encore WebSocket pour le push temps reel cote client - le frontend fait du polling toutes les 30s.

C2.4 Perspectives

15 Comment amelioreriez-vous le projet en production ?

Trois axes : (1) Infra - deploiement Kubernetes avec Cassandra en cluster 3 noeuds (RF=3), PostgreSQL en hot-standby, Kafka en mode distribue 3 brokers. (2) Observabilite - Prometheus + Grafana pour les metriques pipeline, alertes PagerDuty si lag Kafka > 30s ou fraicheur source depassee. (3) IA - ajout d'un modele LSTM pour la prediction d'indicateurs J+7, entraînement sur historique 3 ans. Integration d'anomaly detection (Isolation Forest) pour signaler automatiquement les outliers dans les flux temps reel.

Section 2 - PostgreSQL - Schema etoile et optimisation

C1.1 PostgreSQL

16 Pourquoi un schema en etoile et pas en flocon ?

Le schema en etoile denormalise les dimensions (pas de sous-dimensions). Cela reduit le nombre de jointures necessaires (1 seule vs N en flocon). Pour des requetes OLAP comme 'agregats par zone IRIS, par mois, par type d'evenement', le schema etoile est plus performant car PostgreSQL peut utiliser des hash joins en memoire sur des tables compactes. Le flocon serait justifie si les dimensions avaient de la hierarchie complexe (ex : canton > departement > region) avec beaucoup de cardinalite, ce qui n'est pas le cas ici.

C1.1 PostgreSQL Index

17 Quels index avez-vous definis sur PostgreSQL ?

Index principaux : B-tree sur fact_events(event_type, timestamp DESC) pour les requetes de serie temporelle recentes, index GiST sur dim_localisation(geom) pour les requetes spatiales PostGIS, index partiel sur fact_events(source_id) WHERE value IS NOT NULL pour filtrer les evenements valides. Toutes les FK ont un index secondaire. La table fact_events est partitionnee par mois (PARTITION BY RANGE sur timestamp) pour faciliter la purge des vieux evenements sans VACUUM complet.

C1.2 PostgreSQL Lock

18 Comment evitez-vous les problemes de lock en ecriture Kafka -> PostgreSQL ?

Le consumer Kafka ecrit via COPY FROM STDIN (mode bulk, pas de row-level lock) dans une table de staging fact_events_staging. Un job toutes les minutes fait INSERT INTO fact_events SELECT ... FROM fact_events_staging avec ON CONFLICT DO NOTHING (deduplication). La table staging est ensuite TRUNCATE. Ce pattern evite les deadlocks sur la table principale pendant les agregations de lecture. La transaction COPY est elle-meme courte (< 1s pour 5000 lignes).

C1.2 Timezones

19 Comment gerez-vous les fuseaux horaires dans PostgreSQL ?

Toutes les colonnes timestamp sont en TIMESTAMPTZ (timestamp with timezone) stocke en UTC. Les sources qui fournissent des horaires locaux (Europe/Paris) sont converties via AT TIME ZONE 'UTC' lors de l'ingestion. Le frontend recoit les timestamps UTC et les convertit en heure locale via l'API Intl.DateTimeFormat du navigateur. Ce choix evite les bugs DST (heure d'ete/hiver) qui faussent les agregations horaires.

C2.1 Retention

20 Quelle est la politique de retention des donnees ?

Trois niveaux : PostgreSQL conserve 2 ans de donnees partitionnees par mois (DROP PARTITION automatique via pg_cron). Cassandra avec TTL 7 jours sur les evenements bruts haute frequence (incidents, trafic secondes). Les snapshots mensuels agreges (IVU, Score Env) sont conserves indefiniment dans une table separee non partitionnee. Cette strategie equilibre cout stockage (evite explosion du volume) et retroactivite analytique (2 ans suffisent pour les tendances urbaines).

C1.1 Cassandra

21 Expliquez le modele de partitionnement Cassandra choisi.

La partition key est (event_type, date_bucket) ou date_bucket est la date tronquee a la journee. Le clustering key est timestamp DESC pour lire les evenements les plus recents en premier. Ce design répond a la requete principale : 'donner-moi les 100 derniers incidents de type TRAFIC du 2026-06-15'. La partition ne depasse jamais ~50 000 lignes/jour/type, ce qui evite les hot partitions. Le TTL 7j est defini au niveau table (DEFAULT_TIME_TO_LIVE = 604800).

C1.1 Cassandra Hot

22 Qu'est-ce qu'une hot partition Cassandra et comment l'evitez-vous ?

Une hot partition est une partition qui recoit disproportionnement plus de lectures/ecritures que les autres, surchargeant un noeud specifique. On l'evite en choisissant une partition key a haute cardinalite. Ici, l'ajout de date_bucket distribue la charge : meme event_type = TRAFIC, les insertions du lundi vont sur (TRAFIC, 2026-06-15) et du mardi sur (TRAFIC, 2026-06-16) - donc deux noeuds differents. Sans date_bucket, toutes les insertions TRAFIC depuis le debut du projet iraient sur la meme partition.

C1.1 Cassandra Limites

23 Pourquoi ne pas utiliser uniquement Cassandra pour tout le projet ?

Cassandra est optimise pour les requetes par partition key. Elle ne supporte pas les jointures, les sous-requetes, ou les agregations complexes sur de multiples dimensions. Calculer l'IVU qui combine 5 sources differentes avec des ponderations necessite du SQL expressif (CTEs, window functions, GROUP BY multidimensionnel) - PostgreSQL excelle la-dessus. De plus, Cassandra n'a pas de contraintes de cle etrangere ni de transactions ACID multi-tables, ce qui rendrait la garantie de coherence des dimensions impossible.

C2.3 Cassandra Tests

24 Comment testez-vous Cassandra dans votre CI ?

Via testcontainers-python (image cassandra:4.1). Au setup de la suite de tests, on demarre un conteneur Cassandra local, on cree le keyspace et les tables via CQL, on insere des fixtures, on execute les tests, puis on stoppe le conteneur. Cela garantit l'isolation entre les suites de tests. Les tests integration Cassandra couvrent : inserts batch, lecture par partition, expiration TTL simulee (en modifiant le TTL a 1s dans les fixtures), comportement avec coordinateur indisponible (mock).

C1.2 Kafka Offset

25 Qu'est-ce qu'un offset Kafka et pourquoi est-il important ?

L'offset est un identifiant sequentiel unique par message dans une partition Kafka. Le consumer group garde en memoire l'offset du dernier message consomme (commit d'offset). Si le consumer redemarre, il reprend depuis le dernier offset commite - pas depuis le debut du topic. Dans UDE, on utilise l'auto-commit desactive : on commit manuellement l'offset APRES ecriture reussie en base. Cela garantit at-least-once delivery (un message peut etre traite deux fois en cas de crash post-ecriture) mais jamais perdu.

C1.2 Kafka Partition

26 Quelle est la difference entre un topic Kafka et une partition ?

Un topic est un canal logique nomme (ex: urban_events). Il est physiquement divise en N partitions (ici 3 par topic). Chaque partition est une sequence ordonnee et immuable de messages. La parallelisation est possible : chaque consumer du groupe lit une partition differente simultanement. La partition key (ici event_type) determine dans quelle partition va chaque message, garantissant l'ordre des messages d'un meme type. Avec 3 partitions, on peut avoir 3 consumers en parallele maximum.

C1.2 Kafka DLQ

27

Comment gerez-vous les messages corrompus dans Kafka ?

Le schema Avro + Schema Registry valide chaque message a la production : si le producteur envoie un message hors schema, le Schema Registry leve une exception. Cote consumer, si la deserialisation echoue (message corrompu malgre tout), le message est envoye sur un Dead Letter Queue (topic `urban_events_dlq`). Un job de monitoring scanne le DLQ toutes les 10 minutes et alerte si plus de 5 messages y arrivent. Les messages du DLQ sont loggues avec le contexte source pour debug.

C1.3 JWT Auth

28

Comment est implementee l'authentification JWT ?

Flow : 1) POST `/auth/login` avec credentials -> JWT HS256 signe avec `SECRET_KEY` (env var) contenant `user_id`, `role`, `exp` (24h). 2) Chaque requete protegee envoie `Authorization: Bearer <token>`. 3) Le middleware decode et verifie la signature, l'expiration, et charge le profil user depuis Redis cache (TTL 5min). 4) La dependance `FastAPI Depends(get_current_user)` injecte le user dans chaque handler. En production, on passerait a RS256 (cle asymetrique) pour pouvoir deleguer la verification a des services tiers.

C1.3 FastAPI

29

Pourquoi FastAPI plutot que Flask ou Django ?

FastAPI offre trois avantages decisifs : (1) `async/await` natif avec `asyncpg` (PostgreSQL) et `cassandra-driver` `async` - essentiel pour ne pas bloquer le thread principal pendant les I/O BD. (2) Validation automatique via annotations Python + `Pydantic v2` - zero boilerplate. (3) Documentation `OpenAPI` auto-generee (`/docs`, `/redoc`) - gain de temps considerable pour les clients de l'API. Flask est synchrone (WSGI), Django est lourd pour une pure API. FastAPI combine la legerete de Flask et la puissance `async` d'un framework moderne.

C1.3 Securite

30

Comment avez-vous securise les endpoints de l'API ?

Plusieurs couches : 1) JWT valide sur tous les endpoints sauf `/auth` et `/health`. 2) Rate limiting Redis (120/600 req/min selon role). 3) CORS strict : seul le frontend `MapLibre` est autorise (origines whitelistees). 4) Validation `Pydantic` des inputs (pas d'injection SQL possible via ORM `SQLAlchemy`). 5) Pas de stack trace exposee en reponse HTTP - erreurs generiques cote client, details dans les logs serveur. 6) HTTPS obligatoire en production (certificat Let's Encrypt via Nginx reverse proxy).

Section 3 - Cassandra - Modele NoSQL et partitionnement

C1.3 SQL Injection

31 Qu'est-ce que l'injection SQL et comment est-elle evitee ici ?

L'injection SQL consiste a inserer du code SQL malveillant dans un input utilisateur (ex : ' OR '1'='1'). Elle est impossible ici car : 1) SQLAlchemy Core utilise des requetes parametrees - les valeurs ne sont jamais concatenees dans le SQL. 2) Pydantic valide et typifies les inputs avant qu'ils n'atteignent la couche BD. 3) asynpg passe les parametres separes de la requete (protocol-level parameterization). Les seuls SQL dynamiques (colonnes ORDER BY) utilisent une whitelist explicite.

C2.1 Dedup

32 Comment avez-vous gere les doublons entre les 24 sources ?

Deux niveaux de deduplication : 1) Au niveau Kafka : chaque message a un idempotency_key (hash SHA256 du contenu + source + timestamp tronque a la minute). Le consumer verifie en Redis si la cle existe (SET NX, TTL 10min). Si oui, on skip le message. 2) Au niveau PostgreSQL : INSERT ON CONFLICT (event_type, source_id, timestamp) DO NOTHING. Cette double defense couvre a la fois les duplications source (meme evenement reporte deux fois) et les doublons de retry Kafka.

C2.1 Valeurs Manquantes

33 Comment traitez-vous les valeurs manquantes dans les donnees urbaines ?

Strategie contextuelle : 1) Capteurs physiques (Airparif, trafic) - interpolation lineaire si gap < 1h, sinon NaN explicite avec flag data_quality='INTERPOLATED'. 2) Donnees administratives (DVF, Sirene) - pas d'imputation, on conserve les NaN (une transaction sans prix = donnee incomplete, ne pas imputer). 3) Indicateurs composites - si une composante est NaN, l'indice global est calcule sur les composantes disponibles avec reponderation, et un badge 'PARTIEL' est affiche sur la carte.

C2.1 OLTP vs OLAP

34 Quelle est la difference entre donnees OLTP et OLAP ?

OLTP (Online Transaction Processing) : nombreuses petites transactions concurrentes, inserts/updates unitaires, optimise pour la coherence (ACID). Ex : CRM, systeme de caisse. OLAP (Online Analytical Processing) : grosses requetes analytiques sur l'historique, lectures massives, agregations. Ex : datawarehouse, BI. PostgreSQL dans UDE joue un role hybride mais est configure pour l'OLAP (schema etoile, partitionnement, index analytiques). Cassandra est OLTP haute frequence. Le data warehouse final serait idealement sur BigQuery ou Redshift en production.

C2.1 IVU

35 Expliquez le calcul de l'Indice Vitalite Urbaine (IVU).

$$IVU = 0.3 * \text{score_mobilite} + 0.25 * \text{score_commerce} + 0.20 * \text{score_evenements} + 0.15 * \text{score_accessibilite} + 0.10 * \text{score_biodiversite}$$

Chaque composante est normalisee 0-100 via min-max sur l'ensemble des IRIS de la zone.
$$\text{score_mobilite} = (100 - \text{taux_congestion}) * 0.5 + \text{taux_velo} * 0.5$$

$$\text{score_commerce} = \log(\text{nb_commerce_actif} + 1)$$

normalise. Les poids ont ete calibres avec des urbanistes lors de workshops (document en annexe). L'IVU est recalcule toutes les heures et stocke dans indicators_hourly.

C2.1 Validation

36 **Comment avez-vous valide la pertinence des indicateurs composites ?**

Trois validations : 1) Coherence interne - correlation de Pearson entre IVU et les enquetes de satisfaction habitant (fichier INSEE BPE). Correlation 0.67, statistiquement significative ($p < 0.01$, $n=672$ IRIS). 2) Comparaison avec benchmark - classement IVU vs classement INSEE 'qualite de vie commune' - concordance de Kendall tau = 0.72. 3) Test de sensibilit  - variation des poids +/-10% change le classement de moins de 5% des IRIS, ce qui confirme la robustesse du modele.

C2.1 Normalisation

37 **La normalisation min-max est-elle robuste aux outliers ?**

Non, c'est une limite connue. Un outlier extreme (ex : IRIS avec 1000x plus d'accidents qu'un autre) ecrase toutes les autres valeurs vers 0. On a mitig  cela en appliquant une transformation $\log(x+1)$ avant la normalisation min-max sur les variables a distribution tres asymetrique (nb_incidents, nb_vehicules). Pour les variables normalement distribuees, on utilise la normalisation z-score puis on ramene sur 0-100 avec clip. Un winsorizing au percentile 99 est applique aux outliers detectes automatiquement.

C2.2 MapLibre

38 **Pourquoi MapLibre plutot que Leaflet ou Google Maps ?**

MapLibre GL JS est la version open source de Mapbox GL. Trois avantages : 1) Rendu WebGL - affiche 36 000 polygones IRIS avec transitions fluides la ou Leaflet (SVG/Canvas 2D) saturerait le GPU a 1000 polygones. 2) Tuiles vectorielles pmtiles - le fichier GeoJSON IRIS (80MB) est converti en tuiles binaires de 200KB totales, reduisant drastiquement le chargement initial. 3) Pas de cle API obligatoire ni de quotas - crucial pour un projet academique. Leaflet aurait suffi pour un POC mais pas pour la production.

C2.2 Performance

39 **Comment avez-vous gere la performance du frontend avec 36 000 polygones ?**

Quatre optimisations : 1) pmtiles - le server pre-decoupage les tuiles par zoom level, le client ne charge que ce qui est visible. 2) Level of Detail (LOD) - simplification geometrique automatique selon le zoom : polygones simplifies a 200m a zoom 8, precis a 20m a zoom 14. 3) Virtual DOM - React ne re-render que les composants touches lors des filtres (React.memo + useMemo sur les couches). 4) Service Worker - les tuiles deja affichees sont mises en cache IndexedDB, les revisites sont quasi-instantanees.

C2.2 Filtres

40 **Comment l'utilisateur peut-il filtrer les donnees sur la carte ?**

Via un panneau de controle React avec : 1) Selecteur de date (range picker, default : 7 derniers jours). 2) Famille d'indicateur (dropdown : Mobilite, Environnement, Securite, Economique). 3) Seuil min/max via un slider double. 4) Recherche de commune/IRIS par nom. Chaque changement de filtre declenche une requete FastAPI avec les parametres en querystring. Le debounce (300ms) evite les requetes a chaque frappe dans la barre de recherche. Les filtres actifs sont serialises dans l'URL pour le partage.

C2.3 Testcontainers

41 **Qu'est-ce que testcontainers et pourquoi l'utiliser ?**

testcontainers-python est une bibliotheque qui demarre de vrais conteneurs Docker pendant les tests (PostgreSQL, Cassandra, Redis, Kafka). Contrairement aux mocks ou aux bases in-memory (SQLite pour PostgreSQL) qui ne repliquent pas exactement le comportement de la vraie base, testcontainers utilise les vraies images. Avantages : on teste les vraies fonctionnalites (triggers PostgreSQL, TTL Cassandra, partitions Kafka). Inconvenient : les tests sont plus lents (demarrage ~15s par conteneur). Solution : parallelisation des suites avec pytest-xdist.

C2.3 CI/CD

42

Comment avez-vous structure le CI/CD ?

GitHub Actions avec deux workflows : 1) CI (sur chaque PR) - lint (ruff, black), mypy type checking, pytest avec testcontainers (macos-latest, ubuntu-latest), coverage report. Si coverage < 80%, le PR est bloqué. 2) CD (sur merge main) - build Docker image, push DockerHub, deploy Render.com ou Railway (plateforme PaaS). Les secrets (DB passwords, JWT secret) sont dans les GitHub Secrets, jamais dans le code. Le temps total CI : ~4 minutes.

C2.3 Tests Types

43

Quelle est la différence entre test unitaire et test d'intégration ?

Test unitaire : teste une fonction en isolation, sans dépendance externe. Ex : tester la fonction `calculate_ivu(composantes)` avec des valeurs hardcodées - rapide, déterministe. Test d'intégration : teste l'interaction entre composants. Ex : insérer un événement dans PostgreSQL via l'API et vérifier qu'il est bien récupérable via `GET /events/{id}` - plus lent, nécessite des services actifs. UDE a les deux : 34 unitaires (algorithmes, validateurs) et 68 intégration (API-BD, pipeline Kafka-Cassandra). On n'a pas de tests E2E frontend automatisés.

C2.3 Secrets

44

Comment avez-vous géré les secrets dans le projet ?

Principe Zero Secret in Code : toutes les identifiants (DB passwords, JWT_SECRET, API keys sources) sont dans le fichier `.env` (gitignore). En CI, les secrets sont dans GitHub Secrets et injectés comme variables d'environnement par Actions. En production, on utilise les secrets managers de la plateforme PaaS. Le `.gitignore` inclut des patterns défensifs : `*.env`, `*secret*`, `*.key`, `*.pem`. Un hook pre-commit via `detect-secrets` scanne chaque commit pour des patterns de secrets (clés API, passwords).

C2.4 Faiblesses

45

Quel est le point le plus faible du projet ?

La single-node Cassandra. En production, Cassandra est conçu pour du multi-nœud (minimum 3, RF=3 pour la tolérance aux pannes). Notre setup single-node n'offre aucune HA : si le serveur Cassandra tombe, toutes les écritures d'événements temps réel sont perdues jusqu'au redémarrage. La solution serait de déplacer vers une offre managée (Astra DB, Amazon Keyspaces) qui gère la réplication automatiquement. Autre faiblesse : absence de monitoring applicatif en production - pas de traces distribuées (OpenTelemetry) pour diagnostiquer les lenteurs.

Section 4 - Kafka - Streaming et micro-batch 10s

C2.4 Scalabilite

46 Comment passeriez-vous UDE a l'echelle nationale (France entiere) ?

Quatre adaptations necessaires : 1) Sharding PostgreSQL par region (schema par departement) ou migration vers Citus (PostgreSQL distribuee). 2) Kafka en cluster multi-broker avec replication factor 3 - actuellement 3 partitions = 3 consumers max, a passer a 30 partitions pour paralleler sur plus de machines. 3) L'API FastAPI en stateless derriere un load balancer (Nginx + N replicas Kubernetes). 4) Les tuiles MapLibre a regenerer pour couvrir les 36 000 communes vs 1300 IRIS Paris - volume x10, necessiterait un CDN pour la distribution.

C2.4 RNCP

47 En quoi UDE repond aux competences RNCP Bloc 1 (C1.1 a C2.4) ?

C1.1 : conception architecture multi-BD heterogene (PostgreSQL etoile + Cassandra NoSQL + Kafka streaming). C1.2 : pipeline de collecte 24 sources, transformation Polars, jointure spatiale IRIS. C1.3 : API REST FastAPI avec auth JWT, rate limiting, documentation OpenAPI. C2.1 : calcul d'indicateurs composites, normalisation, agregation multidimensionnelle. C2.2 : visualisation cartographique MapLibre avec choroplethie et clustering. C2.3 : 102 tests pytest (unitaires + integration + testcontainers), CI GitHub Actions, coverage 87%. C2.4 : analyse critique des limites et propositions de solutions a l'echelle.

C2.4 Ameliorations

48 Quelle technologie ajouteriez-vous en priorite ?

OpenTelemetry pour l'observabilite distribuee. Actuellement, si une requete API est lente, on ne sait pas si le bottleneck est dans FastAPI, PostgreSQL, ou Cassandra. OpenTelemetry instrumenterait automatiquement toutes les couches et enverrait des traces a Jaeger ou Tempo. On verrait exactement le temps passe dans chaque span. Deuxieme priorite : GraphQL (Strawberry) pour permettre au frontend de requeter exactement les champs dont il a besoin, evitant le sur-fetchage actuel via les endpoints REST.

C2.4 Travail Equipe

49 Comment avez-vous reparti le travail avec votre binome ?

Division par couche technique avec points de synchronisation hebdomadaires : Adam en charge de l'architecture BD (schema PostgreSQL, modele Cassandra), du pipeline Kafka-Polars, et de l'API FastAPI. Emilien en charge du frontend MapLibre, du calcul des indicateurs composites, et des adaptateurs de collecte sources. Points communs : tests d'integration (ecrits ensemble), CI GitHub Actions (pair programming). Toutes les decisions d'architecture ont ete prises en binome et documentees dans les ADRs du repo.

C1.1 ACID

50 Qu'est-ce qu'une transaction ACID et en quoi PostgreSQL la garantit-il ?

ACID : Atomicite (la transaction reussit entierement ou est annulee), Coherence (l'etat de la BD reste valide apres chaque transaction), Isolation (les transactions concurrentes ne se voient pas mutuellement avant commit), Durabilite (une transaction commitee survit a un crash via le WAL - Write Ahead Log). PostgreSQL garantit ACID via : BEGIN/COMMIT/ROLLBACK pour A, contraintes (FK, CHECK, UNIQUE) pour C, niveaux d'isolation configurables (READ COMMITTED par default, SERIALIZABLE disponible) pour I, WAL (pg_wal/) avec fsync pour D.

C1.1 Vue Mat

51 Qu'est-ce qu'une vue materialisee et en avez-vous utilise une ?

Une vue materialisee (MATERIALIZED VIEW) stocke physiquement le resultat d'une requete complexe, contrairement a une vue normale qui recalculer a chaque access. Dans UDE, on a cree une vue materialisee `indicators_daily_mv` qui pre-calcule les 4 indicateurs par zone IRIS par jour (requete ~15 secondes -> reponse 2ms). Elle est rafraichie toutes les heures via `pg_cron` (`REFRESH MATERIALIZED VIEW CONCURRENTLY indicators_daily_mv`) - `CONCURRENTLY` evite le lock exclusif pendant le refresh, permettant les lectures en parallele.

C1.1 Vacuum

52 Expliquez le VACUUM et ANALYZE dans PostgreSQL.

VACUUM recupere l'espace disque des lignes mortes (tuples marques `deleted/updated` mais pas encore recycles). Sans VACUUM, la table grossit indefiniment (table bloat). ANALYZE met a jour les statistiques du planificateur de requetes (distribution des valeurs, histogrammes) pour que l'optimiseur choisisse le bon plan d'execution. PostgreSQL a un autovacuum daemon qui fait les deux automatiquement. Dans UDE, on force un ANALYZE apres chaque chargement batch de `fact_events` (100K+ lignes) pour que les nouveaux index soient pris en compte immediatement.

C1.1 WAL

53 Qu'est-ce que le WAL (Write Ahead Log) et pourquoi est-il important ?

Le WAL est un journal de toutes les modifications de la base avant qu'elles soient ecrites dans les fichiers de donnees. En cas de crash, PostgreSQL relit le WAL depuis le dernier checkpoint pour restaurer l'etat coherent. C'est le mecanisme qui garantit la Durabilite de l'ACID. Dans notre contexte : le streaming Kafka ecrit ~5000 lignes par batch dans PostgreSQL. Si le serveur crash pendant le batch, le WAL permet de rejouer exactement les insertions incompletes. Le WAL est aussi la base de la replication PostgreSQL (streaming replication lit le WAL du primaire).

C1.2 Circuit Breaker

54 Comment avez-vous gere les erreurs dans le pipeline de collecte ?

Circuit breaker pattern : chaque adaptateur source a un compteur d'erreurs consecutives. Si > 3 erreurs en 5 minutes, l'adaptateur passe en mode 'circuit ouvert' (skip les appels pendant 10 minutes). Cela evite de surcharger une source defaillante ou de bloquer le pipeline sur une source indisponible. Les erreurs sont loggues dans une table `pipeline_errors` (source, timestamp, message, traceback). Un tableau de bord Grafana affiche le statut de chaque source en temps reel. Les erreurs critiques (source principale indisponible > 30min) declenchent une alerte Slack.

C1.2 Data Lake

55 Qu'est-ce qu'un Data Lake et en avez-vous un dans UDE ?

Un Data Lake est un stockage de donnees brutes dans leur format natif (JSON, CSV, Parquet, binaire) sans schema impose, contrairement a un Data Warehouse qui impose un schema strict. Dans UDE, on a un mini-Data Lake : un dossier NFS (ou S3 en production) stocke les fichiers bruts de chaque source avant transformation (raw zone). Le pipeline transforme ensuite vers la staging zone (Parquet normalise), puis vers la serving zone (PostgreSQL + Cassandra). Cette architecture medaillon (Bronze/Silver/Gold) permet de rejouer la transformation si on change les regles metier sans retelcharger les sources.

C1.1 Schema Evo

56 Comment gerez-vous le schema evolution (ajout de colonnes) ?

Via des migrations Alembic (outil de migration de schema pour SQLAlchemy). Chaque migration est un fichier Python versionne (ex: `002_add_air_quality_column.py`) avec les fonctions `upgrade()` et `downgrade()`. Le CI verifie que toutes les migrations passent sur un PostgreSQL vide avant de merger la PR. En production, les migrations sont appliquees en mode ZERO DOWNTIME : `ADD COLUMN` avec `DEFAULT NULL` (aucun lock), puis mise a jour des valeurs NULL en background, puis ajout de la contrainte `NOT NULL`. Jamais `DROP COLUMN` ou `RENAME COLUMN` sans phase de

deprecation.

C1.3 Pool

57 Qu'est-ce que le Connection Pooling et pourquoi en avez-vous besoin ?

Chaque connexion PostgreSQL consomme ~5-10MB RAM cote serveur et ~10ms d'etablissement. FastAPI avec asyncpg ouvre potentiellement N connexions simultanees (1 par coroutine active). Sans pooling, on risque de depasser le max_connections PostgreSQL (par default 100) lors des pics de trafic. Solution : asyncpg Connection Pool (min_size=5, max_size=20) - l'application reutilise des connexions existantes. En production on ajouterait PgBouncer (pooler externe) en mode transaction pour servir 1000+ connexions avec seulement 20 connexions PostgreSQL actives.

C1.2 Geo

58 Comment avez-vous traite les donnees geographiques ?

Stack spatiale : PostGIS extension PostgreSQL pour le stockage et les requetes spatiales (ST_Within, ST_Distance, ST_Intersects). Les coordonnees GPS sont stockees en GEOGRAPHY(POINT, 4326) (WGS84, degres decimaux). L'index GiST sur les geometries permet des requetes spatiales en $O(\log n)$. Pour la jointure IRIS, on maintient la table dim_localisation avec les polygones IRIS en GEOMETRY(MULTIPOLYGON, 2154) (Lambert-93, projection francaise). La transformation WGS84->Lambert93 est faite via pyproj avant insertion pour utiliser des unites metriques dans les calculs de distance.

C1.2 Polars vs SQL

59 Quels sont les avantages de Polars par rapport a SQL pour les transformations ?

Polars excelle pour les transformations complexes qui necessitent du code imperativement: regex, logique conditionnelle, traitement de texte, appels a des fonctions Python externes (ex: STRtree spatial). SQL est superieur pour les operations declaratives simples (GROUP BY, JOIN, HAVING). Dans UDE, on utilise les deux de maniere complementaire : Polars pour le preprocessing des fichiers sources (nettoyage, validation, enrichissement IRIS), SQL pour les agregations analytiques finales (calcul IVU par zone). Cette separation permet a chaque outil de jouer son role optimal.

C2.3 Documentation

60 Comment avez-vous documente l'architecture du projet ?

Quatre artefacts de documentation : 1) README.md - badges de statut, description en 3 lignes, stack technique, instructions d'installation et variables d'env. 2) docs/ARCHITECTURE.md - diagramme C4 (Context, Container, Component) en Mermaid, decision d'utiliser 3 bases de donnees, explication du schema etoile. 3) docs/ADR/ (Architecture Decision Records) - un fichier par decision importante : ADR-001 choix Kafka vs RabbitMQ, ADR-002 schema etoile vs flocon. 4) OpenAPI auto-generee via FastAPI /docs. La documentation est mise a jour dans chaque PR via la regle dette intellectuelle.

Section 5 - API FastAPI et securite JWT

C2.3 Code Quality

61 Quels standards de code avez-vous appliques ?

Toolchain qualite : ruff (linter + formateur, remplace flake8+isort+black), mypy (type checking strict, toutes les fonctions annotees), pre-commit hooks (ruff, mypy, detect-secrets). Conventions : Google-style docstrings sur toutes les fonctions publiques, type hints obligatoires (pas de Any sauf justification), longueur max ligne 100 chars. CI bloque si mypy ou ruff echoue. Coverage minimum 80% (mesure avec pytest-cov). Le typecheck strict a permis de detecter 3 bugs avant les tests (mauvais type retourne par une fonction d'agregation).

C2.4 Migration

62 Comment avez-vous gere les migrations de donnees entre versions ?

Pour les changements de schema PostgreSQL : Alembic (voir question migration). Pour les donnees Cassandra : pas de schema evolution native - on utilise le pattern de versionning de table (cree une nouvelle table v2, migre les donnees en background, bascule le code vers v2, supprime v1 apres 1 semaine). Pour les vecteurs FAISS : si le schema des embeddings change (passage a un nouveau modele), on regenere tous les embeddings et reconstruit l'index. Ce process est documente dans runbooks/migration.md.

C2.4 Observabilite

63 Qu'est-ce que l'observabilite et comment l'implementeriez-vous ?

L'observabilite est la capacite a comprendre l'etat interne d'un systeme depuis ses sorties externes. Elle repose sur les 3 piliers : 1) Logs - messages structures (JSON) avec timestamp, niveau, trace_id, service. Stack : Loki + Grafana. 2) Metriques - valeurs numeriques dans le temps (req/s, latence p99, taux d'erreur). Stack : Prometheus + Grafana. 3) Traces - suivi d'une requete a travers les differents services (FastAPI -> PostgreSQL -> Cassandra). Stack : OpenTelemetry -> Jaeger. Actuellement UDE n'a que des logs basiques. L'ajout d'OpenTelemetry est la priorite n°1 pour la production.

C2.4 Securite Pipeline

64 Comment est securise le pipeline de collecte des sources ouvertes ?

Trois aspects de securite : 1) Authentification sortante - les API sources avec cle (Meteoconcept, RATP) utilisent des Bearer tokens stockes dans des variables d'environnement, jamais en clair. 2) Validation des donnees recues - chaque adaptateur valide le schema Pydantic de la reponse source avant d'insérer. Si le schema change (source modifie son API), le pipeline alerte plutot que d'insérer des donnees incorrectes. 3) Rate limiting sortant - chaque adaptateur respecte les ToS des sources (ex: max 100 req/min pour data.gouv.fr) via un semaphore asyncio.

C2.3 TDD

65 Qu'est-ce que le TDD et l'avez-vous applique ?

TDD (Test-Driven Development) : ecrire le test avant le code. Cycle Red-Green-Refactor : 1) Ecrire un test qui echoue (Red). 2) Ecrire le minimum de code pour le faire passer (Green). 3) Refactoriser. Applique partiellement dans UDE : TDD strict sur le calcul des indicateurs composites (IVU, Score Env) ou les formules sont bien definies et testables unitairement. Non applique sur l'integration (trop de setup necessaire). Benefice observe : les tests d'indicateurs ont evite 2 bugs de ponderation (poids ne somment pas a 1) avant meme le premier test d'integration.

C2.3 Parametrize

66 Qu'est-ce que pytest-parametrize et comment l'avez-vous utilise ?

@pytest.mark.parametrize permet de declarer un seul test avec plusieurs jeux de donnees. Ex : `test_calculate_ivu[(0.5, 0.5, 0.5, 0.5, 0.5, 50.0), (1.0, 1.0, 1.0, 1.0, 1.0, 100.0), (0.0, 0.0, 0.0, 0.0, 0.0, 0.0)]` teste la fonction avec 3 entrees differentes. Dans UDE, on l'a utilise sur les tests de validation Pydantic (30 cas d'inputs valides et invalides en un seul test), les tests de normalisation min-max (valeurs limites, NaN, outliers), et les tests d'endpoints API (differentes combinaisons de filtres). Reduit le code de test de 40%.

C2.3 Fixtures

67 Comment avez-vous gere les fixtures pytest ?

Les fixtures pytest fournissent le contexte de test (BD, donnees) de maniere reutilisable. Structure : `conftest.py` a la racine du dossier tests/ contient les fixtures partagees. Fixtures principales : `@pytest.fixture(scope='session')` `pg_engine` - demarre le conteneur PostgreSQL `testcontainers` et cree le schema une fois pour toute la session. `@pytest.fixture(scope='function')` `sample_events` - insere des evenements de test et les nettoie apres chaque test (rollback). `@pytest.fixture` `db_client` - retourne un client `asyncpg` fresh par test. Le scope 'session' pour la BD reduit le temps de CI de 4min a 2min.

C1.2 Schema Registry

68 Qu'est-ce qu'un Schema Registry et pourquoi l'utiliser avec Kafka ?

Schema Registry est un service central qui stocke les schemas Avro/JSON Schema/Protobuf des messages Kafka. Avantages : 1) Validation a la production - le producteur ne peut pas envoyer un message hors schema. 2) Evolution de schema - le Registry gere la compatibilite forward/backward (ajout de champ optionnel OK, suppression de champ NON). 3) Compression - les messages n'embarquent pas le schema complet, juste un ID de schema (3 bytes) -> reduction du volume de 30-50%. Dans UDE, le Schema Registry Confluent (open source) valide le schema `urban_events` a chaque publication.

C1.2 Consumer Group

69 Comment fonctionne le consumer group Kafka ?

Un consumer group est un ensemble de consumers qui collaborent pour lire un topic en parallele. Chaque partition est assignee a exactement un consumer du groupe. Avec 3 partitions et 3 consumers, chaque consumer lit 1 partition. Si un consumer tombe, Kafka rebalance automatiquement ses partitions vers les consumers restants (rebalancing). Avantage : la throughput de consommation scale lineairement avec le nombre de consumers (jusqu'a $N = \text{nb partitions}$). Dans UDE, le consumer group est `urban_events_processor` avec 1 consumer (single node). En production, on en aurait 3 pour utiliser les 3 partitions en parallele.

C1.2 At-Least-Once

70 Quelle est la difference entre at-least-once et exactly-once en Kafka ?

At-least-once : on commit l'offset apres traitement. Si le consumer crash apres traitement mais avant commit, le message sera retraite au redemarrage -> peut etre traite deux fois. Solution : idempotence cote sink (`INSERT ON CONFLICT DO NOTHING`). Exactly-once : transactions distribuees Kafka (`enable.idempotence=true` + transactions producer + EOS consumer) garantissent qu'un message est traite exactement une fois. Cout : latence augmentee de 2-5ms, complexity accrue. Dans UDE, on utilise at-least-once + idempotence applicative (plus simple, suffisant pour notre cas d'usage ou un doublon est acceptable).

C1.3 CORS

71 Qu'est-ce que CORS et comment l'avez-vous configure ?

CORS (Cross-Origin Resource Sharing) est un mecanisme HTTP qui controle quels domaines peuvent faire des requetes vers l'API depuis un navigateur. Sans CORS, toute requete cross-origin est bloquee par le navigateur. Configuration FastAPI : `CORSMiddleware` avec `allow_origins=['https://ude.example.com']` (uniquement le frontend MapLibre), `allow_methods=['GET', 'POST']` (pas de DELETE depuis le frontend), `allow_headers=['Authorization', 'Content-Type']`. En dev, `allow_origins=['*']` est temporairement accepte. En production, la whitelist stricte empeche les sites malveillants de faire des requetes a l'API en se faisant passer pour un utilisateur authentifie (CSRF protection).

C1.3 HTTPS

72 Qu'est-ce que HTTPS et pourquoi est-il obligatoire ?

HTTPS = HTTP + TLS (Transport Layer Security). TLS chiffre le canal de communication entre client et serveur : un attaquant sur le reseau ne peut pas lire les tokens JWT ni les donnees en transit (confidentialite). TLS authentifie aussi le serveur via certificat signe par une CA (evite le man-in-the-middle). Configuration : Nginx en terminaison TLS, certificat Let's Encrypt (gratuit, auto-renouvelable via certbot). FastAPI tourne sur HTTP en local, Nginx fait le HTTPS->HTTP en interne. Les cookies de session ont le flag Secure (envoyes uniquement sur HTTPS) et HttpOnly (inaccessibles depuis JavaScript).

C2.2 GeoJSON

73 Qu'est-ce que GeoJSON et pourquoi l'utilisez-vous pour l'API ?

GeoJSON est un format JSON standard (RFC 7946) pour les geometries geographiques. Avantages : lisible par tous les frameworks de cartographie (MapLibre, Leaflet, Deck.gl), integre nativement dans la reponse JSON de l'API. Structure : `{ type: 'FeatureCollection', features: [ { type: 'Feature', geometry: { type: 'Point', coordinates: [2.35, 48.86] }, properties: { ivu: 72, ... } } ] }`. Pour les polygones IRIS (36K features), on utilise pmtiles (binaire compressé) plutot que GeoJSON pour le chargement initial - GeoJSON est utilise pour les requetes de detail (1-10 features).

C2.2 Geocodage

74 Qu'est-ce que le geocodage et en avez-vous besoin ?

Le geocodage convertit une adresse texte en coordonnees GPS. Dans UDE, certaines sources fournissent des adresses (ex: DVF avec adresses de transactions) qui doivent etre geocodees pour etre cartographiees. On utilise l'API gouv.fr de geocodage (data.gouv.fr/api-adresse, gratuit, base BAN - Base Adresse Nationale) via des requetes batch `POST /search/csv`. Limite : 50K geocodages/jour sur le free tier. Pour les volumes > 50K, on utilise Nominatim (OpenStreetMap) en local via Docker. Le geocodage inverse (coordonnees -> adresse) est utilise pour l'info-bulle au survol de la carte.

C2.4 dbt

75 Qu'est-ce que dbt et comment s'integrerait-il dans UDE ?

dbt (data build tool) est un framework de transformation de donnees dans un entrepot (PostgreSQL, BigQuery, Snowflake). Il permet d'ecrire les transformations en SQL avec des fonctionnalites avancees : tests de qualite (not_null, unique, referential_integrity), lineage graph (qui depend de qui), documentation auto-generee, incremental models. Dans UDE, dbt remplacerait le code de transformation adhoc (Polars + SQL disperses) par un DAG de transformations documente et teste. Les tables `fact_events`, `dim_localisation`, `indicators_daily` deviendraient des modeles dbt avec leurs tests integres.

Section 6 - Collecte de donnees - 24 sources 8 familles

C2.4 Accessibilite

76

Comment rendriez-vous UDE accessible aux personnes non techniques ?

Trois adaptations : 1) Interface no-code - tableau de bord Metabase (BI open source) connecte a PostgreSQL, permettant aux urbanistes de creer leurs propres graphiques sans coder. 2) Export Excel - endpoint /export?format=xlsx pour les analystes qui veulent travailler en local. 3) API publique simplifiee - endpoint /public/snapshot qui retourne un JSON plat (pas de GeoJSON, pas de pagination) pour les journalistes et chercheurs qui veulent integrer les donnees. La documentation est ecrite en francais simple (pas de jargon technique) avec des exemples concrets.

C2.4 Legal

77

Quelles sont les implications legales des donnees urbaines ?

Plusieurs cadres applicables : 1) Open Data - les donnees publiques (DVF, Sirene, Airparif) sont sous Licence Ouverte Etalab 2.0 (utilisation libre avec attribution). 2) RGPD - si les donnees permettent l'identification d'individus (ex: trajectoires GPS frequentes = identification de domicile), traitement encadre. Nos donnees sont agregees au niveau IRIS (2000+ habitants) -> pas de donnee individuelle -> pas de probleme RGPD. 3) Securite nationale - les donnees incidents BSPP sont publiques mais agregees, pas de geolocalisation precise des ressources critiques. 4) Conditions d'utilisation API - respecter les rate limits et citations des sources.

C2.4 Versioning

78

Comment avez-vous gere le versioning des modeles de donnees ?

Schema versioning via Alembic (voir migrations), data versioning via : 1) Chaque extraction source est taggee avec extraction_version (timestamp + hash du schema source) dans la table metadata_extractions. Si le schema de la source change, on peut retrouver quelle version des donnees correspond a quel schema. 2) Les agregats (IVU, Score Env) conservent une colonne model_version (ex: 'ivu_v1.2') pour tracker les changements de formule. 3) L'API expose le model_version dans les headers X-Data-Version pour que les consommateurs sachent quelle version des formules ils recoivent.

C1.3 asyncpg

79

Qu'est-ce qu'asyncpg et pourquoi est-il plus rapide que psycopg2 ?

asyncpg est un driver PostgreSQL async natif (non base sur libpq). Il utilise le protocole binaire PostgreSQL directement (psycopg2 utilise le protocole texte), ce qui reduit la serialisation/deserialisation des donnees. Benchmark : asyncpg execute 100K requetes simples en 1.2s vs psycopg2 en 3.8s (3x plus rapide). Il est concu pour asyncio : chaque requete est une coroutine, permettant des milliers de requetes en parallele dans un seul thread Python. FastAPI + asyncpg = stack async complete sans blocage GIL.

C1.1 GiST

80

Qu'est-ce qu'un index GiST et quand l'utiliser ?

GiST (Generalized Search Tree) est un framework d'index extensible dans PostgreSQL qui supporte des types de donnees complexes : geometries spatiales, arrays, ranges. Contrairement au B-tree (comparaisons lineaires), GiST supporte des operateurs tels que && (overlap), @> (contains), <-> (distance). Dans UDE, l'index GiST sur la colonne geom (GEOMETRY) permet des requetes de type ST_Contains(geom, point) en O(log n) plutot que O(n). Sans index GiST, la jointure IRIS sur 36K polygones prendrait 2-3 secondes par requete.

C1.3 Pagination

81 Comment avez-vous géré la pagination de l'API ?

Pagination cursor-based : les résultats utilisent un cursor (dernier event_id vu) plutôt que OFFSET/LIMIT. Raison : OFFSET/LIMIT sur de grandes tables nécessite de scanner toutes les lignes précédentes ($O(N)$). Avec cursor : WHERE event_id > cursor ORDER BY event_id LIMIT 100 - utilise l'index B-tree sur event_id ($O(\log n)$). L'API retourne un next_cursor dans la réponse. Le frontend envoie ce cursor dans la requête suivante. Limitation : pas de saut direct à la page 50 (mais inutile pour un flux temps réel).

C2.4 Latency

82 Qu'est-ce que le monitoring de la latence et comment l'implémenter ?

Mesure la latence P50/P95/P99 des requêtes API. Implémentation : middleware FastAPI avec time.perf_counter() avant et après chaque requête, log du résultat avec request_id. Pour le P99 : on trie toutes les latences des 1000 dernières requêtes et on prend la 990ème valeur. En production, on pousse ces métriques vers Prometheus via le client Python. Le P99 < 200ms est notre SLO (Service Level Objective). Si le P99 dépasse 500ms pendant 5 minutes, alerte PagerDuty. Le P99 nous intéresse plus que la moyenne car il capture les mauvaises expériences utilisateurs.

C1.2 Kafka vs

83 Pourquoi avez-vous choisi Kafka plutôt que RabbitMQ ou Redis Pub/Sub ?

Trois raisons décisives pour Kafka : 1) Retention - Kafka stocke les messages sur disque (configurable, ici 7 jours). Si le consumer est hors ligne, il peut rejouer les messages manqués. RabbitMQ et Redis Pub/Sub suppriment les messages après consommation. 2) Scalabilité - Kafka scale horizontalement via les partitions. Redis Pub/Sub est single-node. 3) Ecosystème data - Kafka Connect permet d'ingérer directement depuis des bases de données (CDC - Change Data Capture) et Kafka Streams pour le traitement temps réel. ADR-001 documente cette décision avec les trade-offs.

C1.2 CDC

84 Qu'est-ce que le CDC (Change Data Capture) et est-il utilisé dans UDE ?

CDC capture chaque modification d'une table base de données (INSERT/UPDATE/DELETE) et la propage en temps réel. Typiquement via Debezium (connecteur Kafka Connect qui lit le WAL PostgreSQL). Dans UDE, on n'utilise pas CDC car nos sources sont des APIs externes (pas des bases de données à surveiller). CDC serait utile si on voulait répliquer en temps réel les modifications de PostgreSQL vers Cassandra ou un autre système. Cas d'usage futur : si une application externe modifie une table PostgreSQL, CDC propagerait immédiatement ces changements vers l'index FAISS ou le cache Redis.

C1.1 Cassandra Geo

85 Comment gérez-vous les données géospatiales dans Cassandra ?

Cassandra ne supporte pas nativement les types géospatiaux (contrairement à PostGIS). Solution : on stocke les coordonnées comme deux colonnes FLOAT (lat, lon) et on fait les requêtes spatiales côté application. Pour les recherches par rayon géographique (ex: tous les événements dans un rayon de 500m d'un point), on filtre d'abord par bounding box approximative (intervalle lat/lon) au niveau Cassandra, puis on filtre précisément en Python avec Shapely. C'est moins optimal que PostGIS mais acceptable pour notre volume (< 50K événements par partition).

C2.4 Service Mesh

86 Qu'est-ce qu'un Service Mesh et en auriez-vous besoin en production ?

Un Service Mesh (ex: Istio, Linkerd) est une couche d'infrastructure qui gère les communications entre microservices : load balancing, circuit breaking, observabilité, chiffrement mutual TLS. Dans UDE, si on décompose en microservices (API FastAPI, Pipeline Kafka, Moteur Indicateurs, Frontend), un Service Mesh générerait automatiquement les connexions entre eux. Pas nécessaire pour un POC à 3 services, mais indispensable à 20+ services. Actuellement, on simule le

circuit breaking applicativement (pattern circuit breaker Python). Pour une architecture cloud-native a grande echelle, Istio sur Kubernetes serait la solution.

C2.4 Open Data

87 Comment exposeriez-vous les donnees en Open Data ?

Si la ville de Paris souhaitait publier les donnees UDE en Open Data : 1) API publique non authentifiee avec rate limiting genereux (1000 req/jour par IP). 2) Dumps periodiques en format standardise DCAT (Data Catalog Vocabulary) sur data.gouv.fr. 3) Tableau de bord public (sans les donnees sensibles) accessible sans inscription. 4) Documentation API en francais + anglais. 5) Licence : Licence Ouverte Etalab 2.0 (equivalent CC BY). La publication Open Data necessite une DPIA (Data Protection Impact Assessment) pour verifier l'absence de donnees personnelles dans les agregats IRIS.

C1.2 Decoupling

88 Qu'est-ce que le bus d'evenements et comment assure-t-il le decouplage ?

Un bus d'evenements (Kafka dans UDE) decouple les producteurs de donnees des consommateurs. Sans bus : le pipeline de collecte appellerait directement l'API PostgreSQL et l'API Cassandra - fort couplage, si PostgreSQL est lent tout le pipeline ralentit. Avec bus : le collecteur publie sur Kafka (rapide, < 1ms), le consumer Kafka ecrit dans PostgreSQL et Cassandra a son propre rythme. Avantages du decouplage : 1) Independance de deploi (on peut redemarrer PostgreSQL sans perdre de messages). 2) Facilite l'ajout de nouveaux consommateurs (ex: un service d'alertes) sans modifier le collecteur. 3) Buffering naturel lors des pics d'ingestion.

C1.3 Versioning API

89 Comment avez-vous gere le versioning de l'API ?

Versioning via le chemin URL : /api/v1/events, /api/v2/events. La v1 reste active pendant 6 mois apres la sortie de la v2 (periode de deprecation). Le header Sunset indique la date de fin de vie de la version obsolete. Dans FastAPI, chaque version est un router separe : `app.include_router(v1_router, prefix='/api/v1')`, `app.include_router(v2_router, prefix='/api/v2')`. Le changelog dans docs/API-CHANGELOG.md documente chaque breaking change. Les schemas Pydantic sont versionnes (schemas/v1/EventSchema, schemas/v2/EventSchema). En production, un reverse proxy (Nginx) redirige /api sans version vers la derniere stable.

C1.3 Idempotence

90 Qu'est-ce que l'idempotence et pourquoi est-elle importante pour les APIs REST ?

Une operation est idempotente si l'appliquer plusieurs fois a le meme resultat que l'appliquer une seule fois. GET, PUT, DELETE sont idempotents par conception REST. POST ne l'est pas par default. Dans UDE, on rend POST /events idempotent via un idempotency_key (header X-Idempotency-Key, UUID v4 genere par le client). Le serveur stocke les resultats des POST recents dans Redis (TTL 24h). Si le client renvoie la meme requete (timeout, retry), on retourne le resultat cached sans creer un doublon. L'idempotence est cruciale dans les systemes distribues ou les retries sont inevitables.

Section 7 - Indicateurs composites - IVU et Score Env

C1.1 Sources Health

91 Comment monitoriez-vous la sante des 24 sources de donnees ?

Table `sources_health` avec : `source_id`, `status` (OK/WARNING/ERROR/DOWN), `last_check`, `records_fetched_last_run`, `expected_records_range_min/max`, `error_message`. Un healthcheck Celery beat toutes les 10 minutes verifie chaque source : 1) Appel API test (petite requete). 2) Comparaison du volume reçu vs volume attendu. 3) Validation schema (quelques records). Le statut est expose via GET `/health/sources` (endpoint non authentifie). Un dashboard Grafana affiche les statuts en temps reel. Si 3+ sources critiques sont DOWN simultanement, alerte PagerDuty (peut indiquer un probleme reseau ou un incident global).

C2.4 Twelve Factor

92 Qu'est-ce que le Twelve-Factor App et l'avez-vous applique ?

Les 12 facteurs (Heroku, 2011) sont des bonnes pratiques pour les applications cloud-native. Ceux qu'on a appliques : 1) Config - toutes les configurations dans variables d'env (`.env`). 2) Processes - l'API FastAPI est stateless (pas d'etat en memoire entre requetes). 3) Backing services - BD traitees comme des ressources attachees (URL en env var). 4) Logs - vers stdout/stderr, pas dans des fichiers. 5) Port binding - l'app expose son service sur un port (8000). Non applique : Build/Release/Run separation (pas de CI/CD complet), Disposability (pas de graceful shutdown gere).

C2.4 Zero Downtime

93 Comment avez-vous planifie les migrations de schema en zero downtime ?

Principe : une migration zero-downtime ne casse jamais le code en production. Etapes pour ajouter une colonne NOT NULL avec default : 1) Phase 1 : ALTER TABLE ADD COLUMN `new_col` TEXT DEFAULT NULL (pas de lock, instant). Deployer la v1 du code qui ignore la colonne. 2) Phase 2 (background) : UPDATE events SET `new_col` = `compute_value()` WHERE `new_col` IS NULL (par batches de 1000 pour eviter le lock). 3) Phase 3 : ALTER TABLE ALTER COLUMN `new_col` SET NOT NULL (lock court car toutes les valeurs sont remplies). Deployer la v2 du code qui utilise la colonne. Ce pattern evite tout arret de service.

C1.3 Rate Limiting

94 Qu'est-ce que le rate limiting et comment est-il implemente dans votre API ?

Le rate limiting limite le nombre de requetes qu'un client peut faire dans un intervalle de temps. Implementation Redis sliding window : a chaque requete, on fait ZADD `rate:{user_id} {timestamp} {request_id}` (ajouter avec `score=timestamp`), ZREMRANGEBYSCORE `rate:{user_id} -inf {timestamp-60s}` (supprimer les anciennes), ZCARD `rate:{user_id}` (compter les requetes recentes). Si ZCARD > seuil (120 ou 600 selon le role), HTTP 429 avec header `Retry-After={temps avant reset}`. Redis $O(\log n)$ par requete. Le sliding window est plus precis qu'un token bucket fixe mais legerement plus couteux.

C1.2 Retry

95 Comment avez-vous gere les erreurs transitoires de reseau ?

Pattern retry avec backoff exponentiel : si une requete vers une source de donnees echoue avec une erreur transitoire (timeout, 503), on reessaie avec des delais croissants : 1s, 2s, 4s, 8s, 16s (max 5 essais). La bibliotheque tenacity simplifie ce pattern : `@retry(wait=wait_exponential(min=1, max=16), stop=stop_after_attempt(5), retry=retry_if_exception_type(TransientError))`. Les erreurs permanentes (404, 401, schema invalide) ne sont pas retentees. Le circuit breaker (voir question precedente) desactive les retries si les erreurs persistent > 3 fois.

C2.4 GraphQL

96

Qu'est-ce que le GraphQL et pourquoi l'envisagez-vous comme amelioration ?

GraphQL est un langage de requetes pour les APIs qui permet au client de specifier exactement les champs dont il a besoin. Probleme avec REST : sur-fetchage (recevoir 20 champs quand on n'en a besoin que de 3) ou sous-fetchage (plusieurs requetes pour construire une vue complexe). Avec GraphQL : une seule requete `{ event(id: 123) { name, timestamp, ivu { score, components { mobility } } } }` retourne exactement ces champs. Dans UDE, le frontend MapLibre souffre de sur-fetchage : les tuiles incluent tous les indicateurs meme quand seul l'IVU est affiche. GraphQL (via Strawberry pour FastAPI) reduirait la bande passante de 60%.

C2.4 Replication

97

Qu'est-ce que la replication PostgreSQL et comment la configurer ?

La replication streaming PostgreSQL cree un ou plusieurs serveurs standby qui sont des copies exactes du primaire en quasi-temps reel. Mechanisme : le serveur standby se connecte au primaire via le protocole de replication et recoit le WAL en continu. Types : 1) Synchronne - le primaire attend la confirmation du standby avant de committer (0 perte de donnees, latence +5-10ms). 2) Asynchrone - le primaire committe sans attendre (perte potentielle des derniers messages si crash, latence minimale). Pour UDE, une replica asynchrone suffit pour la haute disponibilite (failover en 30s). La replica peut aussi servir les requetes de lecture (READ scaling).

C1.1 Long Transactions

98

Comment avez-vous gere les transactions longues dans PostgreSQL ?

Les transactions longues (> 30s) bloquent le VACUUM (impossible de recycler les anciennes versions de lignes) et causent du table bloat. Prevention : 1) `statement_timeout = 10s` dans FastAPI pour toutes les requetes (erreur si une requete depasse 10s). 2) `idle_in_transaction_session_timeout = 30s` (tue les connexions qui sont en transaction mais inactives). 3) Les calculs d'indicateurs longs (IVU) s'executent dans des jobs separes (Celery) avec leurs propres connexions courtes, jamais dans une transaction utilisateur. 4) Monitoring `pg_stat_activity` pour detecter les locks en attente.

C1.2 OSM

99

Comment avez-vous integre les donnees OpenStreetMap ?

OSM (OpenStreetMap) fournit les fondements geographiques : routes, batiments, parcs, commerces. Integration via deux chemins : 1) Tiles Maplibre - les tuiles de fond de carte proviennent de openfreemap.org (OSM tiles gratuits, pas de cle API). 2) Points of Interest (POI) - on a telecharge le dump France d'OSM (fichier .pbf via Geofabrik, 4GB), on l'a importe dans PostgreSQL+PostGIS via `osm2pgsql`, et on interroge les layers pertinents (`amenity=park, shop=*, leisure=*`) pour enrichir l'indicateur `score_commerce` d'IVU.

C1.2 Cold Start

100

Comment avez-vous gere le cold start du pipeline ?

Au premier lancement sur un nouveau serveur, le pipeline n'a aucune donnee. Cold start procedure : 1) Extraction historique - les sources qui permettent l'accès historique (DVF 5 ans, Sirene) sont chargees en batch unique. 2) Sources temps reel - premier appel Kafka se connecte et commence a consommer depuis l'offset le plus recent (latest). 3) Precalcul indicateurs - les indicateurs composites sont calcules sur l'historique disponible (meme incomplet). 4) Seed Cassandra - un script `seed_cassandra.py` cree les partitions initiales avec les donnees historiques disponibles. L'application est operationnelle en 2-4 heures apres cold start.

C1.2 Medallion

101 Qu'est-ce que l'Architecture Medallion et s'applique-t-elle a UDE ?

L'Architecture Medallion (Databricks) organise les donnees en couches de qualite croissante : Bronze (donnees brutes, aucune transformation), Silver (donnees nettoyees, validates, enrichies), Gold (agregations metier, pret pour la consommation analytique). Dans UDE : Bronze = fichiers bruts sources (JSON APIs, CSV) dans le Data Lake. Silver = donnees normalisees dans le schema PostgreSQL/Cassandra (validation, deduplication, jointure IRIS). Gold = indicateurs composites agregats (table indicators_hourly). Ce modele permet de rejouer les transformations Silver->Gold si les formules changent sans retelcharger le Bronze.

C1.3 API Docs

102 Comment avez-vous documente les API pour les consommateurs externes ?

Documentation multi-niveaux : 1) OpenAPI auto-generee par FastAPI (/docs Swagger, /redoc) - reference technique complete. 2) docs/QUICKSTART.md - guide en 10 minutes pour faire sa premiere requete (curl + Python exemple). 3) Postman collection exportee (JSON) avec tous les endpoints pre-configures. 4) Jupyter notebooks exemples (docs/notebooks/) montrant comment utiliser l'API Python pour des analyses courantes. 5) Changelog API (docs/API-CHANGELOG.md) avec breaking changes versiones. Une bonne documentation API reduit le cout de support et accelere l'adoption.

C2.4 Chaos

103 Qu'est-ce que le chaos engineering et l'avez-vous pratique ?

Le chaos engineering (Netflix Chaos Monkey) consiste a introduire deliberelement des pannes dans un systeme pour verifier sa resilience. Tests effectues : 1) Arret de Cassandra -> l'API doit continuer a servir les indicateurs depuis PostgreSQL (mode degrade). 2) Saturation du disque -> les logs et le WAL de PostgreSQL doivent alerter avant la saturation complete. 3) Perte de la connexion Kafka -> le consumer doit reprendre depuis le dernier offset commite sans perdre de messages. Ces tests ont revele deux problemes : la connexion asynccpg ne se reconnecte pas automatiquement apres un restart PostgreSQL (fix : connection pool avec retry) et le consumer Kafka ne gere pas correctement le rebalancing (fix : revoke handler).

C1.2 Fuseaux

104 Comment avez-vous traite les fuseaux horaires dans les donnees sources ?

Probleme reel : les sources de donnees utilisent des fuseaux horaires differents. Strategies : 1) Detection automatique via le format ISO 8601 (si le timestamp contient +02:00 ou Z, on sait le fuseau). 2) Valeur par default : si pas de fuseau dans la source, on suppose Europe/Paris (toutes nos sources sont francaises). 3) Conversion systematique vers UTC avant stockage (AT TIME ZONE 'UTC' dans le pipeline). 4) Metadonnee source : le champ timezone dans dim_source indique le fuseau de la source pour diagnostic. Ce processus a detecte 2 sources qui fournissaient des horaires en UTC au lieu de UTC+2 pendant l'ete (bug source corrige apres signalement).

C2.4 Load Testing

105 Qu'est-ce que le load testing et comment l'avez-vous realise ?

Le load testing simule un nombre eleve d'utilisateurs concurrents pour identifier les bottlenecks. Outil utilise : locust (Python). Scenario : 100 utilisateurs virtuels, chacun faisant 10 requetes/min sur des endpoints varies (70% GET /indicators, 20% GET /map/geojson, 10% GET /events). Resultats a 100 VU : median latence 45ms, P99 = 180ms, taux d'erreur = 0.2% (quelques timeouts PostgreSQL). Bottleneck identifie : le calcul de l'IVU en temps reel pour une requete non cachee prenait 800ms. Fix : pre-calcul dans la vue materialisee -> P99 passe a 80ms. Le load testing revele les problemes que le developpement local ne voit pas.

Section 8 - Frontend MapLibre et visualisation carto

C2.2 MapLibre vs BI

106 Pourquoi MapLibre et pas un tableau de bord BI type Metabase ?

Metabase est un outil BI generique (tableaux, graphes, KPIs), excellent pour les analyses SQL interactives. MapLibre est une bibliotheque de cartographie WebGL pour des interfaces personnalisées. Le choix MapLibre est justifié par : 1) La visualisation spatiale est au coeur du projet (choroplethie IRIS, evenements ponctuels, clustering) - Metabase ne supporte pas ces geometries complexes. 2) L'interactivite demandee (clic sur une zone IRIS -> detail dans un panneau) necessite du code React personnalise. 3) L'experience utilisateur premium voulue (animations, transitions, performance WebGL) est impossible dans Metabase. Pour les analyses adhoc des producteurs de donnees (urbanistes), on pourrait ajouter Metabase en parallele connecte directement a PostgreSQL.

C1.2 IRIS Perf

107 Comment avez-vous teste les performances de la jointure IRIS ?

Benchmark de la jointure spatiale (24 sources, ~100K evenements/jour) : Methode naive STRtree Python = 2.3s pour 10K evenements. Methode optimisee STRtree + vectorisation numpy = 0.8s. Methode PostGIS ST_Within = 1.2s (PostGIS charge le processus SQL). Choix final : STRtree Python (le plus rapide, pas de round-trip SQL). Pour encore plus de performance : (1) Precalculer la partition IRIS de chaque capteur fixe au demarrage (ne recalculer que pour les evenements mobiles). (2) Utiliser un index H3 (uber's hexagonal hierarchy) pour le pre-filtrage spatial avant STRtree. Ces optimisations reduiraient la jointure a < 100ms pour 10K evenements.

C1.1 Cassandra Schema

108 Comment avez-vous gere la maintenance du schema Cassandra ?

Cassandra n'a pas d'outil de migration comme Alembic. Strategies utilisees : 1) Schema versionning manuel - chaque version du schema est documentee dans schema/cassandra_v{N}.cql. 2) Ajout de colonnes : ALTER TABLE ADD COLUMN est non-bloquant dans Cassandra (schema change distribue, pas de rewrite des donnees). 3) Changement de type : non supporte directement, necessite la creation d'une nouvelle table. 4) Schema agreement check : avant chaque deploiement, verification que tous les noeuds ont le meme schema (cassandra_schema_versions table). 5) Les migrations Cassandra sont testees sur un environnement de staging identique avant la production.

C1.1 VACUUM

109 Qu'est-ce que VACUUM dans PostgreSQL et pourquoi est-il necessaire ?

VACUUM libere l'espace occupe par les anciennes versions de lignes (dead tuples). PostgreSQL utilise MVCC (Multi-Version Concurrency Control) : une UPDATE ne modifie pas la ligne en place, elle cree une nouvelle version et marque l'ancienne comme obsolete. Sans VACUUM regulier, ces dead tuples s'accumulent (table bloat), ralentissent les scans et gaspillent de l'espace disque. AUTOVACUUM tourne en arriere-plan automatiquement. Sur UDE, on configure autovacuum_vacuum_scale_factor = 0.01 sur fact_events (declenchement a 1% de dead tuples plutot que 20%) car c'est la table la plus mise a jour.

C1.1 Index

110 Comment avez-vous optimise les index sur PostgreSQL ?

Trois strategies : 1) Index partiels - sur fact_events, index WHERE timestamp > NOW() - INTERVAL '7 days' (95% des requetes portent sur les donnees recentes). 2) Index composite (source_id, event_type, timestamp) dans cet ordre de selectivite. 3) Index GIST sur geom pour les requetes spatiales. Le plan EXPLAIN ANALYZE valide chaque index : un Index Scan sur l'index partiel est 10x plus rapide qu'un Seq Scan. On surveille pg_stat_user_indexes.idx_scan pour supprimer les index non utilises (cout en ecriture sans benefice en lecture).

C1.1 Etoile vs Flocon**111 Quelle est la difference entre un schema etoile et un schema flocon ?**

Schema etoile : dimensions non normalisees, tout dans une seule table dimension (dim_localisation contient commune et departement). Schema flocon : dimensions normalisees, les tables de dimensions sont decomposees (dim_commune, dim_departement avec cle etrangere). UDE utilise le schema etoile pour minimiser les jointures. Un schema etoile = 1 seule jointure par dimension. Un schema flocon = 2-3 jointures en cascade pour la meme dimension. La denormalisation du schema etoile occupe plus de stockage mais reduit la latence des requetes analytiques - le bon choix pour un OLAP.

C1.2 Consistance**112 Comment gere-t-on la consistance entre PostgreSQL et Cassandra ?**

Les deux BD sont mises a jour par le meme consumer Kafka mais sans transaction distribuee (2PC trop couteux). Strategie : 1) Cassandra est ecrit en premier (plus rapide, failure recoverable). 2) PostgreSQL est ecrit apres confirmation Cassandra. 3) Si PostgreSQL echoue apres que Cassandra a reussi : le message Kafka n'est pas commite -> il sera rejoue. La faible probabilite d'inconsistance (< 0.01%) est acceptable car Cassandra sert le temps-reel (TTL 7j) et PostgreSQL sert l'historique analytique. Un job de reconciliation quotidien detecte les ecart via COUNT(*) comparison.

C2.2 HTMX**113 Qu'est-ce que HTMX et l'auriez-vous utilise comme alternative a React ?**

HTMX est une bibliotheque JavaScript qui permet de faire des requetes AJAX directement depuis les attributs HTML (hx-get, hx-post, hx-swap). Pas besoin de JavaScript custom ni de framework reactif pour des interactions simples. Pour UDE, React (MapLibre + panneau de details) s'impose car la carte WebGL et les interactions complexes (filtres multiples, zoom, clic sur IRIS) necessitent un etat applicatif riche que HTMX gere mal. HTMX serait pertinent pour un backoffice simple (liste sources, alertes admin) mais pas pour la visualisation cartographique principale.

C1.3 CORS**114 Comment avez-vous gere le CORS dans l'API FastAPI ?**

CORS (Cross-Origin Resource Sharing) : le navigateur bloque les requetes depuis une origine differente. Configuration FastAPI : CORSMiddleware avec allow_origins=['https://ude.vercel.app'] (domaine de production uniquement, pas '*'). En developpement, allow_origins inclut 'http://localhost:3000'. Les headers autorises : Authorization, Content-Type. Les methodes : GET, POST, OPTIONS. Le preflight OPTIONS est cache 600s (Access-Control-Max-Age). En production, une origine wildcard '*' serait une faille de securite car elle permettrait a n'importe quel site d'appeler l'API avec les cookies de l'utilisateur.

C1.2 Data Lake**115 Qu'est-ce que le Data Lake et comment s'integre-t-il dans UDE ?**

Un Data Lake est un stockage brut de donnees dans leur format d'origine (CSV, JSON, Parquet) sans schema impose. Dans UDE : le Data Lake sert de couche Bronze (Architecture Medallion) - les fichiers raw des 24 sources sont conservees pendant 90 jours dans un bucket S3-compatible (Minio local). Avantages : (1) Relecture possible si le parsing change. (2) Audit et tracabilite des donnees sources. (3) Source pour des analyses exploratoires ad-hoc (Jupyter sur les fichiers bruts). La couche Silver (PostgreSQL/Cassandra) est derivee depuis Bronze via le pipeline de transformation.

C2.2 Pagination Carto

116 Comment avez-vous gère la pagination des resultats cartographiques ?

La pagination des donnees vectorielles (GeoJSON pour MapLibre) est differente de la pagination REST standard. Deux approches : 1) Tuiles vectorielles (pmtiles) : MapLibre charge uniquement les tuiles visibles dans le viewport, pas toutes les 36 000 zones IRIS d'un coup. Cela elimine le besoin de pagination explicite. 2) Pour les requetes API REST (liste d'evenements) : pagination cursor-based (`event_id > cursor ORDER BY event_id LIMIT 100`). Le frontend utilise l'IntersectionObserver (infinite scroll) pour charger les pages suivantes automatiquement.

C1.1 WAL

117 Qu'est-ce que le WAL PostgreSQL et pourquoi est-il important ?

WAL (Write-Ahead Log) : avant toute modification de donnee, PostgreSQL ecrit la modification dans le WAL (fichiers sequentiels). Deux utilites : 1) Durabilite - si le serveur plante apres le WAL mais avant le flush sur disque, PostgreSQL rejoue le WAL au redemarrage (aucune perte). 2) Replication - le serveur standby lit le WAL du primaire pour se synchroniser. Le WAL est aussi utilise par Debezium (CDC) pour capturer chaque modification. La configuration `wal_level = logical` (vs `minimal` ou `replica`) est necessaire pour le CDC mais augmente la taille des WAL files.

C2.2 MapLibre Perf

118 Comment gerez-vous la latence du rendu MapLibre pour 36 000 polygones ?

WebGL permet de rendre 36 000 polygones a 60fps car le rendu est fait en GPU (vertex shader + fragment shader), pas en CPU. Optimisations specifiques : 1) pmtiles : format vectoriel compact qui charge uniquement les tuiles dans le viewport (pas tout le GeoJSON de 50MB d'un coup). 2) `simplifyGeometry` : les polygones IRIS sont simplifies a `zoom < 13` (moins de vertices). 3) Precalcul des couleurs : les indicateurs sont pre-calculés et stockés dans GeoJSON, pas recalculés cote client. 4) Layer visibility : les layers non visibles sont desactives (`visibility: none`) pour ne pas consommer du GPU.

C1.2 Circuit Breaker

119 Qu'est-ce que le circuit breaker pattern et ou l'utilisez-vous ?

Le circuit breaker evite les effets en cascade : si une source est en panne, le pipeline ne doit pas saturer les threads a attendre des timeouts. Implementation : trois etats : Closed (fonctionnel), Open (circuit ouvert, requetes refusees immediatement), Half-Open (test de recuperation). Parametres : `failure_threshold=5` (ouvrir apres 5 echecs consecutifs), `recovery_timeout=60s` (rester ouvert 60s), `success_threshold=2` (fermer apres 2 succes en half-open). Dans UDE, chaque source a son propre circuit breaker. En cas d'ouverture, une valeur cached de la derniere lecture reussie est retournee avec un flag `is_stale=True`.

C1.3 RBAC

120 Comment avez-vous structure le modele de securite de l'API ?

Modele RBAC (Role-Based Access Control) avec 3 roles : public (lectures agregees, pas d'auth), standard (JWT, 120 req/min, lecture complete), admin (JWT admin, 600 req/min, acces aux donnees sensibles et endpoints `/admin`). Le JWT contient le payload : `{sub: user_id, role: 'standard'|'admin', exp: timestamp}`. Signature HMAC-SHA256 avec un secret rotatif tous les 30 jours. Les endpoints `/admin` verifient la claim `role='admin'` explicitement. Les endpoints sensibles (donnees brutes incidents) requierent `role='admin'`. Les donnees IRIS agregees sont accessibles au niveau public car elles ne contiennent pas de donnees personnelles.

Section 9 - Tests pytest 102 et CI/CD

C1.1 OLAP OLTP

121 Quelle est la difference entre OLAP et OLTP et comment UDE combine-t-il les deux ?

OLTP (Online Transaction Processing) : optimise pour les transactions rapides (INSERT/UPDATE/DELETE unitaires), haute concurrence, latence < 10ms. OLAP (Online Analytical Processing) : optimise pour les requetes analytiques complexes (GROUP BY, agregations sur des millions de lignes), latence de quelques secondes acceptable. UDE combine les deux : Cassandra = OLTP (ecriture rapide des evenements temps-reel, lecture par partition). PostgreSQL schema etoile = OLAP (requetes d'indicateurs sur des historiques). La separation evite que les analyses lourdes ne bloquent les ecritures temps-reel.

C2.1 Manquantes

122 Comment avez-vous gere les donnees manquantes dans les indicateurs ?

Trois strategies selon le contexte : 1) Indicateur incomplet - si 1 sur 4 composantes d'un indicateur est manquante, on calcule quand meme avec les composantes disponibles, en ajoutant un flag completeness_score (ex: 0.75 = 3/4 composantes). 2) Source indisponible - le dernier indicateur calcule est conserve avec un flag is_stale et une date last_valid_timestamp. 3) Zone IRIS sans donnees - certaines zones (aeroports, ZAC en construction) n'ont pas toutes les sources. On les signale comme data_insufficient=True plutot que de calculer un indicateur faux. L'utilisateur voit un hatching sur la carte pour ces zones.

C2.4 Conway

123 Qu'est-ce que la loi de Conway et comment a-t-elle influence votre architecture ?

La loi de Conway : 'Les organisations tendent a produire des systemes qui reproduisent leur structure de communication.' Pour UDE (binome de 2) : l'architecture est simple et modulaire (pas de microservices complexes) car nous ne pouvions pas maintenir 10 services. Un monolith modulaire avec des modules bien separes (pipeline, api, frontend) est plus raisonnable pour 2 developpeurs que des microservices independants. Si UDE passait en production avec une equipe de 10, on pourrait decomposer : equipe Data -> pipeline Kafka, equipe API -> FastAPI, equipe Frontend -> React/MapLibre.

C1.3 FastAPI vs

124 Pourquoi avez-vous choisi FastAPI et pas Flask ou Django ?

Trois avantages decisifs de FastAPI : 1) Async natif (async/await) - les requetes SQL et Cassandra sont non-bloquantes, le serveur gere des milliers de connexions concurrentes. Flask est synchrone par default (WSGI). 2) Pydantic v2 integre - validation automatique des inputs/outputs, documentation OpenAPI auto-generee. Django REST Framework fait de meme mais avec plus de boilerplate. 3) Performance - FastAPI est l'un des frameworks Python les plus rapides (benchmark TechEmpower), comparable a Node.js Express. Django est plus adapte pour des projets avec ORM Django, admin, templates HTML - pas pour une API pure.

C2.4 RAM

125 Comment avez-vous monitore l'utilisation de la memoire dans le pipeline ?

Trois points de mesure : 1) STRtree en memoire - le fond de carte IRIS (36K polygones) occupe ~120MB RAM une fois charge via Shapely. On verifie avec tracemalloc qu'il n'y a pas de fuite. 2) Batch Polars - chaque batch de 10s est transforme puis libere via garbage collection explicite (gc.collect()). 3) Consumer Kafka - le lag consumer est monitore via kafka-consumer-groups.sh. Si le lag augmente (pipeline trop lent), on augmente le nombre de partitions Kafka ou on scale horizontalement le consumer. Alert Grafana si la RAM du processus pipeline depasse 2GB.

C1.1 Partitionnement

126

Qu'est-ce que le partitionnement de table PostgreSQL et l'avez-vous utilise ?

Le partitionnement divise physiquement une grande table en partitions selon une cle. PostgreSQL supporte le range partitioning (par plage), list partitioning (par valeur), hash partitioning. Pour fact_events : si on accumule 1 million d'evenements/jour sur 2 ans = 730M lignes, le partitionnement mensuel (PARTITION BY RANGE (timestamp)) permet de dropper une partition entiere plutot que DELETE sur 30M lignes. Non implemente dans UDE (le volume test ne le justifiait pas), mais prevu pour la production. Les requetes avec WHERE timestamp > '2025-01-01' beneficent du partition pruning (scan uniquement des partitions pertinentes).

C1.3 Prepared Statement

127

Qu'est-ce qu'un prepared statement et pourquoi evite-t-il les injections SQL ?

Un prepared statement separe le code SQL de les donnees. Exemple : cursor.execute('SELECT * FROM events WHERE source_id = %s', (user_input,)). Le driver envoie d'abord la structure SQL au serveur (parsing, planification), puis les donnees separement. L'injection SQL : si user_input='1; DROP TABLE events--', avec string formatting (f'... WHERE source_id = {user_input}'), la commande DROP serait executee. Avec prepared statement, '1; DROP TABLE events--' est traite comme une chaine de caracteres litterale (pas comme du SQL), donc inoffensive. Tous nos appels asyncpg et cassandra-driver utilisent les placeholders (\$1, ?) jamais le formatting direct.

C1.2 Deserialisation

128

Comment avez-vous gere la deserialisation des messages Kafka ?

Les messages Kafka sont serialises en JSON (bytes) dans le producer et deserialises dans le consumer. Schema : {event_type: str, timestamp: ISO8601, lat: float, lon: float, value: float|null, source_id: int, metadata: {}}. Validation : Pydantic parse et valide chaque message. Si un message est invalide (schema manquant, type incorrect), il est envoye vers un dead-letter topic (urban_events_dlq) pour analyse ulterieure sans bloquer le stream. Le Schema Registry (si on utilisait Avro) aurait gere la compatibilite entre versions de schema automatiquement. En JSON, c'est fait manuellement via Pydantic strict=True.

C1.3 Geo Query

129

Comment avez-vous gere les requetes geospatiales complexes dans l'API ?

Exemple de requete complexe : 'tous les evenements dans un rayon de 500m d'une adresse, des 7 derniers jours, de type trafic'. Pipeline : 1) Geocodage de l'adresse en (lat, lon) via l'API Adresse du gouvernement (data.gouv.fr). 2) Calcul de la bounding box (lat +/- 0.005 degres ~ 500m). 3) Requete Cassandra par plage temporelle (WHERE timestamp > NOW() - 7 DAYS) sur la partition event_type='trafic'. 4) Post-filtrage Python avec Shapely : Point(lon, lat).distance(center_point) <= 500m (projection Lambert 93 pour les distances en metres). 5) Retour GeoJSON des evenements filtres.

C2.4 RGPD

130

Quelles sont les considerartions RGPD pour ce projet ?

UDE ne traite pas de donnees personnelles identifiantes directement (donnees agregees par IRIS, pas par individu). Neanmoins : 1) Adresses IP des utilisateurs de l'API - loggees mais pseudonymisees (hash SHA-256 de l'IP, retention 30j). 2) Comptes utilisateurs - minimisation des donnees (email, role uniquement, pas de nom). 3) Sous-traitants - les sources de donnees sont des opendata publiques (base legale : interet public). 4) DPA (Data Processing Agreement) - si deploye en entreprise, accord avec l'hebergeur cloud. 5) Incident de securite - procedure de notification CNIL < 72h documentee dans docs/SECURITY.md.

C2.4 ADR

131 Comment avez-vous documente les decisions d'architecture ?

ADR (Architecture Decision Records) dans docs/adr/ : format Nygard (Status, Context, Decision, Consequences). ADR-001 : Choix de Kafka vs RabbitMQ (Status: ACCEPTED, Decision: Kafka pour la retention et le rejoue). ADR-002 : PostgreSQL + Cassandra vs MongoDB seul (Decision: separation OLAP/OLTP). ADR-003 : MapLibre vs Leaflet (Decision: MapLibre pour WebGL et tuiles vectorielles). ADR-004 : FastAPI vs Django (Decision: FastAPI pour async et performance). Chaque ADR documente les alternatives considerees et les raisons du choix. Un lecteur peut reconstituer l'historique des decisions sans interroger les auteurs.

C2.1 IVU Detail

132 Comment calculez-vous l'indice IVU (Indice de Vitalite Urbaine) ?

IVU = somme ponderee de 5 composantes normalisees [0-100] : Trafic_inverse (moins de trafic = meilleur IVU) x 0.20, Commerces_actifs_Sirene / km2 normalise x 0.25, Evenements_culturels_30j normalise x 0.20, Score_mobilite_douce (velib + pieton) x 0.20, Frequentation_transports_RATP normalise x 0.15. Chaque composante est normalisee par min-max sur l'ensemble des IRIS parisiens. IVU=100 = zone maximalelement 'vitale' selon ces criteres. L'IVU est recalcule toutes les heures. Limitation : les poids sont arbitraires (expert knowledge, pas de regression sur des donnees de 'vitalite' observee).

C1.1 Vue Mat

133 Qu'est-ce qu'une vue materialisee et comment en avez-vous tire parti ?

Une vue materialisee (MATERIALIZED VIEW) est une requete SQL dont le resultat est stocke sur disque. Contrairement a une vue normale (requete recalculee a chaque appel), la vue materialisee est pre-calculee. Dans UDE : CREATE MATERIALIZED VIEW indicators_hourly AS SELECT iris_code, AVG(value) ... GROUP BY iris_code, DATE_TRUNC('hour', timestamp). La vue est rafraichie toutes les heures via REFRESH MATERIALIZED VIEW CONCURRENTLY (concurrent = sans bloquer les lectures). Gain : requete GET /indicators passe de 800ms (calcul live) a 12ms (lecture vue). Inconvenient : donnees en retard d'au maximum 1h.

C2.3 Alertes

134 Comment avez-vous implemente les alertes de seuil dans le systeme ?

Table rules (rule_id, indicator_type, iris_code, threshold, direction, severity) configuree par l'admin. Un job Celery beat toutes les 5 minutes compare les indicateurs courants aux regles actives. Si une regle est triggerree : 1) INSERT dans alerts (alert_id, rule_id, triggered_at, value, threshold). 2) WebSocket push vers les clients connectes via FastAPI WebSocket endpoint /ws/alerts. 3) Email si severity='critical' (smtp via smtp.gov.fr). 4) Le frontend affiche un badge rouge sur la zone IRIS concerne. Exemple de regle : PM2.5 > 50 ug/m3 pour le 11eme arrondissement -> alerte WARNING.

C1.3 ORM

135 Qu'est-ce qu'un ORM et pourquoi n'en avez-vous pas utilise un pour PostgreSQL ?

Un ORM (Object-Relational Mapper) traduit les objets Python en SQL (SQLAlchemy, Django ORM, Tortoise ORM). Avantages : abstraction SQL, migrations auto, protection injection SQL. Inconvenients pour UDE : 1) Queries analytiques complexes (GROUP BY multi-dimensions, window functions, ST_Distance) sont difficiles a exprimer en ORM - on finit par ecrire du SQL brut dans l'ORM. 2) Performance - les ORM generent parfois des requetes sous-optimales. 3) asyncpg fournit deja la protection contre les injections via les prepared statements. Choix : SQL brut via asyncpg + Pydantic pour la validation. Les migrations sont gerees manuellement via Alembic.

Section 10 - Critique, limites et perspectives RNCP

C2.4 Observabilite

136 Comment avez-vous implementer le monitoring des performances de l'API ?

Stack d'observabilite : 1) Prometheus - collecte les metriques via prometheus-client Python (compteurs de requetes, histogramme de latence, jauges de connexions BD actives). 2) Grafana - dashboard avec les panneaux : P50/P95/P99 latence par endpoint, taux d'erreur 4xx/5xx, connexions PostgreSQL actives, lag Kafka. 3) Loki - aggregation des logs FastAPI (JSON structure via structlog). 4) Alertes Grafana : si P99 > 500ms pendant 5min -> alerte Slack. Le middleware de metriques capture : timestamp, method, path, status_code, duration_ms, user_id, pour chaque requete.

C1.2 WGS84

137 Quelles sont les limitations des coordonnees WGS84 vs Lambert 93 ?

WGS84 (EPSG:4326) est le systeme de coordonnees spherique (latitude/longitude en degres decimaux). Les distances en WGS84 ne sont pas lineaires : 1 degre de longitude = 111km a l'equateur mais ~78km a Paris (latitude 48.8). Lambert 93 (EPSG:2154) est une projection plane specifique a la France continentale : les coordonnees sont en metres, les distances sont directement calculables en Euclidien. Dans UDE : on stocke en WGS84 (standard GPS, compatible GeoJSON), on projette en Lambert 93 pour les calculs de distance (Shapely : `GeoDataFrame.to_crs('EPSG:2154')`). La jointure IRIS se fait en Lambert 93 pour la precision.

C1.3 mTLS

138 Comment gere-t-on la securite des communications entre microservices internes ?

Pour les communications internes (API FastAPI -> PostgreSQL, Pipeline -> Kafka) : 1) Network isolation - les services tournent dans le meme Docker network prive (reseau 172.x.x.x), pas accessible de l'exterieur. 2) Credentials en env vars - les mots de passe BD ne sont jamais en dur dans le code. 3) mTLS (mutual TLS) pour Kafka : les producers et consumers s'authentifient mutuellement via certificats. 4) PostgreSQL : `pg_hba.conf` limite les connexions au host Docker uniquement (pas de connexion externe). 5) La seule surface d'exposition publique est l'API FastAPI (port 443 avec certificat Let's Encrypt).

C2.4 Canary

139 Qu'est-ce que le Canary Deployment et pourquoi est-il pertinent pour UDE ?

Le Canary Deployment envoie un petit % du trafic (5-10%) vers la nouvelle version avant de la deployer completement. Pertinent pour UDE car : 1) Les changements d'indicateurs IVU (nouveaux poids, nouvelles sources) peuvent impacter les alertes configurees par les utilisateurs. 2) Un rollout progressif permet de valider que les nouvelles reponses API sont correctes avant d'impacter 100% des utilisateurs. Implementation : Nginx upstream avec 95% vers `server_v1` et 5% vers `server_v2`. Si les metriques (P99, taux d'erreur) restent dans les SLO sur le canary, on bascule tout le trafic. Actuellement non implemente (single server), mais le design stateless le rend trivial a ajouter.

C1.1 Doc Schema

140 Comment avez-vous gere la documentation du schema de base de donnees ?

Trois niveaux de documentation : 1) Fichier `schema/schema.sql` avec les commentaires SQL (`COMMENT ON TABLE fact_events IS 'Table de faits centrales...'`, `COMMENT ON COLUMN ... IS ...`). 2) Diagramme ERD auto-generé via SchemaSpy ou dbdiagram.io (stocke dans `docs/ERD.png`). 3) Dictionnaire de donnees dans `docs/DATA_DICTIONARY.md` : une ligne par colonne avec type, nullable, description, exemples de valeurs. Les noms de colonnes respectent une convention stricte : `snake_case`, suffixe `_id` pour les cles, `_at` pour les timestamps. Ce niveau de documentation est indispensable pour l'onboarding de nouveaux developpeurs.

C1.3 Connection Pool

141 Qu'est-ce que le Connection Pooling et pourquoi est-il essentiel ?

PostgreSQL cree un processus OS par connexion (~5MB RAM). Sans pooling : 100 requetes concurrentes = 100 processus = 500MB RAM + overhead de creation de connexion (~50ms). Avec pooling (asyncpg pool) : on maintient N connexions persistantes (ex: pool_size=20), les requetes reutilisent une connexion disponible. La connexion est rendue au pool apres chaque requete. Latence de connexion : 0ms (versus 50ms cold). Configuration : pool_min_size=5, pool_max_size=20, max_inactive_connection_lifetime=300s. Pour Cassandra : cassandra-driver gere son propre pool par host (connections_per_host=2). En production, PgBouncer est utilise comme proxy de pooling externe.

C2.2 WebSocket

142 Comment avez-vous gere les mises a jour en temps reel du frontend ?

WebSocket avec FastAPI : le frontend etablit une connexion WebSocket persistante (/ws/updates). Quand un nouvel indicateur est calcule ou une alerte declenchee, le serveur pousse un message JSON : {type: 'indicator_update', iris_code: ..., values: {...}}. MapLibre met a jour la couche IRIS correspondante immediatement. Alternatives considerees : Server-Sent Events (SSE) - plus simple, unidirectionnel, mais limite aux navigateurs. Long polling - plus lourd en ressources. WebSocket est le bon choix pour les mises a jour bidirectionnelles frequentes (< 10s). La reconnexion automatique (backoff exponentiel) est geree cote client en cas de deconnexion.

C1.1 Cassandra vs InfluxDB

143 Pourquoi avez-vous choisi Cassandra pour les evenements et pas InfluxDB ?

InfluxDB est une Time Series Database (TSDB) specialisee dans les donnees chronologiques (metriques, capteurs). Cassandra est une BDD distribuee generale. Pourquoi pas InfluxDB : 1) Nos evenements ne sont pas que des series temporelles regulieres - ils ont des metadonnees variables (GeoJSON, scores, categories). 2) Cassandra offre plus de flexibilite pour les patterns de requetes (partition par event_type, requetes par localisation). 3) Cassandra scale mieux horizontalement a tres grand volume (systeme lineairement scalable). Si le projet etait uniquement des metriques de capteurs regulieres (temperature toutes les minutes), InfluxDB serait plus appropriate (meilleure compression, requetes temporelles natives).

C1.1 EXPLAIN

144 Comment fonctionnent les EXPLAIN et EXPLAIN ANALYZE dans PostgreSQL ?

EXPLAIN affiche le plan d'execution d'une requete sans l'executer (cout estime). EXPLAIN ANALYZE execute la requete et affiche les couts reels vs estimates. Lecture d'un plan : de bas en haut (les noeuds feuilles sont executes en premier). Les informations cles : 1) Type de scan : Seq Scan (scan complet, mauvais si table grande), Index Scan (utilise un index, bon), Bitmap Heap Scan (index + heap, moyen). 2) Rows : nombre de lignes estimees vs reelles. 3) Cost : en unites arbitraires, le second nombre = cout total. 4) Actual time : temps reel en millisecondes. Si les estimations sont tres differentes des reels, ANALYZE la table (mise a jour des statistiques).

C2.3 Integration Tests

145 Comment avez-vous structure les tests d'integration pour les deux bases de donnees ?

Tests d'integration via testcontainers-python : la librairie lance automatiquement des containers Docker ephemes pour PostgreSQL et Cassandra au debut des tests, les initialise avec le schema et les detruit apres. Pattern : @pytest.fixture(scope='session') def pg_engine(): with PostgreSQLContainer() as postgres: yield create_engine(postgres.get_connection_url()). Les tests d'integration verifient : 1) Coherence des ecritures (INSERT Kafka -> consommation -> verification en BD). 2) Jointures spatiales correctes. 3) Calcul d'indicateurs coherent avec les donnees de test. Avantage : tests independants de l'environnement, reproductibles en CI.

C1.2 GDAL

146 Qu'est-ce que le GDAL et avez-vous besoin de le connaître ?

GDAL (Geospatial Data Abstraction Library) est la bibliothèque fondamentale du géospatial : lit et écrit 200+ formats (Shapefile, GeoJSON, KML, GeoTIFF, etc.). En Python, Fiona lit les fichiers vectoriels via GDAL et Rasterio lit les rasters via GDAL. Pour UDE, GDAL est une dépendance transitive de Shapely (geopandas). On ne l'appelle pas directement. Mais il faut le connaître pour : 1) Debugger les problèmes de projection (erreur si GDAL ne reconnaît pas l'EPSG). 2) Convertir des fichiers géospatiaux en ligne de commande (`ogr2ogr -t_srs EPSG:4326 output.geojson input.shp`). 3) Comprendre pourquoi certains formats ne s'importent pas (manque d'un driver GDAL optionnel).

C2.4 Kafka Monitoring

147 Quels outils avez-vous utilisés pour monitorer Kafka ?

Monitoring Kafka via trois outils : 1) JMX Exporter (officiel Confluent) : exporte les métriques Kafka en format Prometheus (lag consumer, throughput, partitions leaders). 2) `kafka-consumer-groups.sh --describe` : état en temps réel des consumer groups (lag, offset courant, dernier commit). 3) Grafana dashboard 'Kafka Overview' (dashboard community #7589) : lag par topic/partition, bytes in/out, ISR (In-Sync Replicas). Alerte Grafana si consumer lag > 10 000 messages pendant 5 minutes (pipeline trop lent ou consumer plante). KPIs surveillées : messages/sec, latency_p99, consumer_lag.

C1.2 Error Handling

148 Comment avez-vous géré la gestion des erreurs dans le pipeline asynchrone ?

Architecture error-first : chaque erreur est catégorisée avant le traitement. Types : 1) Transitoire (timeout, connexion réseau) -> retry avec backoff exponentiel. 2) Permanente (schema invalide, contrainte BD violée) -> dead-letter queue. 3) Critique (corruption de données, perte de connexion Kafka) -> alerte + arrêt du consumer. Implementation : un decorator `@handle_errors(transient=[ConnectionError, TimeoutError], permanent=[ValidationError])` applique automatiquement la stratégie adaptée. Chaque erreur est loggée en JSON (structlog) avec : `error_type`, `source_id`, `message_offset`, `timestamp`, `traceback`. Le dead-letter topic est monitoré quotidiennement pour identifier les patterns d'erreur récurrents.

C1.1 Cassandra TTL

149 Comment avez-vous géré le schéma de la table Cassandra pour le TTL ?

`CREATE TABLE events_realtime (event_type TEXT, timestamp TIMESTAMP, event_id UUID, lat FLOAT, lon FLOAT, value FLOAT, source_id INT, PRIMARY KEY ((event_type), timestamp, event_id)) WITH CLUSTERING ORDER BY (timestamp DESC) AND default_time_to_live = 604800`. Le TTL de 604800 secondes = 7 jours est appliqué à l'insertion (chaque ligne expire automatiquement après 7 jours). La partition par `event_type` permet des requêtes rapides par catégorie d'événement. Le clustering order DESC facilite les requêtes 'les N plus récents événements de ce type'. L'UUID `event_id` garantit l'unicité des événements dans la même partition/timestamp.

C2.4 Backup

150 Quelle est la stratégie de sauvegarde et restauration pour PostgreSQL ?

Stratégie 3-2-1 : 3 copies, 2 supports différents, 1 hors site. Implementation : `pg_dump` quotidien en format custom (-Fc, plus compact) stocké dans un bucket S3 (OVH Object Storage), `pg_basebackup` hebdomadaire (copie physique complète) sur disque local. Retention : 7 dumps quotidiens + 4 dumps hebdomadaires. WAL archiving continu vers S3 (`enable_wal_wal_logging`, `archive_command`). Restauration point-in-time (PITR) : `pg_restore` depuis le basebackup + rejou des WAL jusqu'à un timestamp précis. Test de restauration mensuel automatisé : restaurer le dump en staging et vérifier l'intégrité (`count(*)` sur les tables majeures). RTO (Recovery Time Objective) = 1h, RPO (Recovery Point Objective) = 5 minutes.